

Application Development

- Individual Coursework Report –

SmartMed Pharmacy System

Module Leader : Ms.Shifani Mohideen

Student : M.N Sahla Baanu – E301118 [25045082]



Course Submission Cover Sheet

Module: Application Development

Assignment no: 001

Weighting: 100% of the module mark

Deadline: 11.59 PM Tuesday the 14th July 2026

Module Leader: Ms.Shifani Mohideen

Student ID: E301118 / 25045082

Please note that there are specific regulations concerning **the use of AI and Academic Misconduct**. Below are extracts from these regulations. By signing, you acknowledge that you have read and understood these extracts.

(signature:) M.N Sahla Baanu Date: 14/07/2026

This header sheet should be attached to the work you submit.

Academic Integrity means being honest in your academic work and your studies and making sure that you acknowledge the work of others and giving credit where you have used other people's ideas as part of presenting your arguments. Your assessment submissions must therefore always be entirely your own work, based on your own learning and appropriately referenced including how you have used Generative AI. The University regards the use of Generative AI applications by students to deceive to gain unfair advantage as **academic misconduct**. This usage includes:

- **Plagiarism**, where AI tools are used to generate output and ideas that are presented or submitted as if they were the student's own work, without proper citation or references.
- Where a complete assignment is created using Generative AI and represented as a student's own work, this will be regarded as contract cheating in the same way as commissioning an 'Essay Mill' or other third party to complete your work. Further information can be found on : [Guidance on the use of Artificial Intelligence](#).

Academic misconduct: The University takes academic misconduct very seriously and seeks at all times to rigorously protect its academic standards. Plagiarism, collusion and other forms of cheating constitute academic misconduct, for which there is an explicit range of graduated penalties depending on the particular type of academic misconduct. The penalties that can be applied if academic misconduct is substantiated range from a reprimand to expulsion in very serious cases and for repeated instances of misconduct. You are also responsible for ensuring that all work submitted is your own and that it is appropriately referenced. The University does not tolerate cheating of any kind. You are strongly advised to familiarise yourself with the Academic Misconduct Policy and Procedure ([Academic Misconduct](#)), which list a range of categories of academic misconduct and associated penalties, covering instances of academic misconduct (plagiarism, collusion, exam cheating).

Table of Contents

Course Submission Cover Sheet	2
Drive Link : Source Code	4
Introduction	5
System Requirements	6
Functional Requirements	6
Admin	6
Customer	6
Additional Features	7
Non-Functional Requirements	8
Hardware Requirements	8
Software Requirements	9
Development Tools and Technologies	9
Design Diagrams	10
Architecture Diagram	10
Use Case Diagram	12
ER Diagram	13
Class Diagram	14
Sequence Diagram	15
Database Design	17
Description of Classes, Properties and Methods	21
User	21
Admin	21
Customer	22
Medicine	23
Order	24
OrderItem	24
Prescription	25
Service Classes	25
Implementation	26
Brief Description of Functions	26

Introduction.....	Error! Bookmark not defined.
Application startup.....	26
Login.....	29
Customer registration.....	31
Forgot password.....	33
Change password	35
Admin dashboard.....	37
Manage medicines	39
Manage customers	40
Manage orders.....	42
Generate reports	44
Customer home dashboard.....	46
Search medicines and add to cart.....	49
Medicine details	50
Place order (checkout)	52
Track orders	53
Profile management	56
Search Algorithms	57
User Manual.....	60
Installation Guide.....	61
Using the Application	62
Reflection.....	79
Conclusion	80

Drive Link : Source Code

https://drive.google.com/drive/folders/10LazhqRwMLgC_APiYrVmpY1_2d6BpjLN?usp=sharing

Introduction

This report presents SmartMed, a pharmacy management system. The application is intended to help a small pharmacy handle routine work through one Windows desktop program instead of depending on paper records or scattered files. Two user types are supported: an admin who manages the pharmacy operation, and a customer who signs in to use the customer-facing side of the system. Both roles enter through the same login screen, but each user sees only the features relevant to that role.

The system was built using C# with Windows Forms on .NET Framework 4.8, which is well suited to a desktop business application of this kind. Persistent data is stored in a SQL Server database, with ADO.NET used to connect the forms to the database through helper and repository classes. The project structure separates the user interface, business logic, and data access, which makes the codebase easier to follow as features are added. Supporting diagrams such as use case, class, and sequence models were prepared to plan and explain how the system behaves.

The overall aim was to apply object-oriented programming in a practical desktop application that a pharmacy could realistically use. Key goals included reliable login for both roles, keeping stock and order information accurate, and giving the admin useful dashboard and report information without manual calculations. Another priority was to keep the code readable and well organised so that further features could be added later without disrupting existing functionality.

System Requirements

Functional Requirements

Admin

- Access the system using secure administrator credentials to perform and manage all pharmacy-related operations.
- Maintain medicine information by adding new medicines, updating existing records, removing discontinued medicines, and managing details such as name, category, dosage, price, stock quantity, supplier information, expiry dates, and prescription requirements.
- Monitor and manage customer information by viewing customer records and updating customer details when necessary.
- Oversee customer orders by viewing all orders placed through the system and updating order statuses from Pending to Ready for Pickup or Delivered.
- Monitor pharmacy performance through a dashboard that provides an overview of total sales, available medicine stock, active orders, and other key operational statistics.
- Generate and review reports related to sales performance, medicine inventory, and customer order history to support decision-making and operational monitoring.
- Create, update, and remove discounts or promotional offers that can be applied to medicines.
- Track medicine expiry dates and receive notifications for medicines approaching expiry in order to ensure inventory safety and compliance.

Customer

- Create a personal account and securely access the system using registered credentials.
- Search and browse available medicines by name, category, or price range and view detailed medicine information before purchasing.
- Select medicines, add them to a shopping cart, review selected items, and place orders through the system.

- Monitor previously placed orders and track their current status throughout the order processing lifecycle.
- Manage personal profile information by viewing and updating personal and contact details.
- Upload prescription documents when purchasing medicines that require prescription verification.
- View available discounts and promotional offers that can be applied to eligible medicines.
- Export personal order history in PDF or Excel format for future reference and record keeping.

Additional Features

- Low Stock Alert System - The system will automatically monitor medicine inventory levels and notify administrators when the stock quantity of a medicine falls below a predefined threshold. This helps prevent stock shortages and ensures the availability of essential medicines.
- Medicine Expiry Warning Dashboard - The system will identify medicines approaching their expiry dates and display warning notifications on the administrator dashboard. This assists in inventory management and reduces the risk of dispensing expired medicines.
- Prescription Verification System - The system will require customers to upload a valid prescription when ordering medicines marked as prescription-required. Orders containing prescription medicines cannot be processed unless the required prescription document has been uploaded and verified.
- Password Change Functionality - The system will allow both administrators and customers to securely change their account passwords after successful authentication, improving account security and user management.
- Order Cancellation - The system will allow customers to cancel orders that are still in the Pending status. Once an order has been processed or marked as Ready for Pickup, cancellation will no longer be permitted.

Non-Functional Requirements

- Usability - The system shall provide an intuitive and user-friendly interface with role-based navigation, allowing administrators and customers to easily access their respective functionalities through clearly organized forms and menus.
- Reliability - The system shall validate user inputs before processing and use parameterized SQL queries to ensure accurate data handling and prevent database-related errors.
- Maintainability - The system shall follow a three-tier architecture with separate Presentation, Business Logic, and Data Access layers to simplify future maintenance, testing, and enhancements.
- Performance - The system shall provide responsive operation for medicine searches, order processing, and report generation, with search and filtering functionality optimized for typical pharmacy inventory sizes.
- Security - The system shall authenticate users through secure login mechanisms, restrict access based on user roles, and maintain session information for authenticated users.
- Availability - The system shall allow authorized users to access pharmacy services whenever the application and database are operational.
- Scalability - The system shall support future enhancements such as additional user roles, advanced reporting features, and expanded inventory management without major architectural changes.
- Data Integrity - The system shall maintain consistent and accurate data through validation rules, database constraints, and controlled update operations.

Hardware Requirements

- PC or laptop running Windows 10 or later.
- Intel Core i3 processor or equivalent.
- Minimum 4 GB RAM (8 GB recommended for development).
- At least 5 GB of available storage space.

Software Requirements

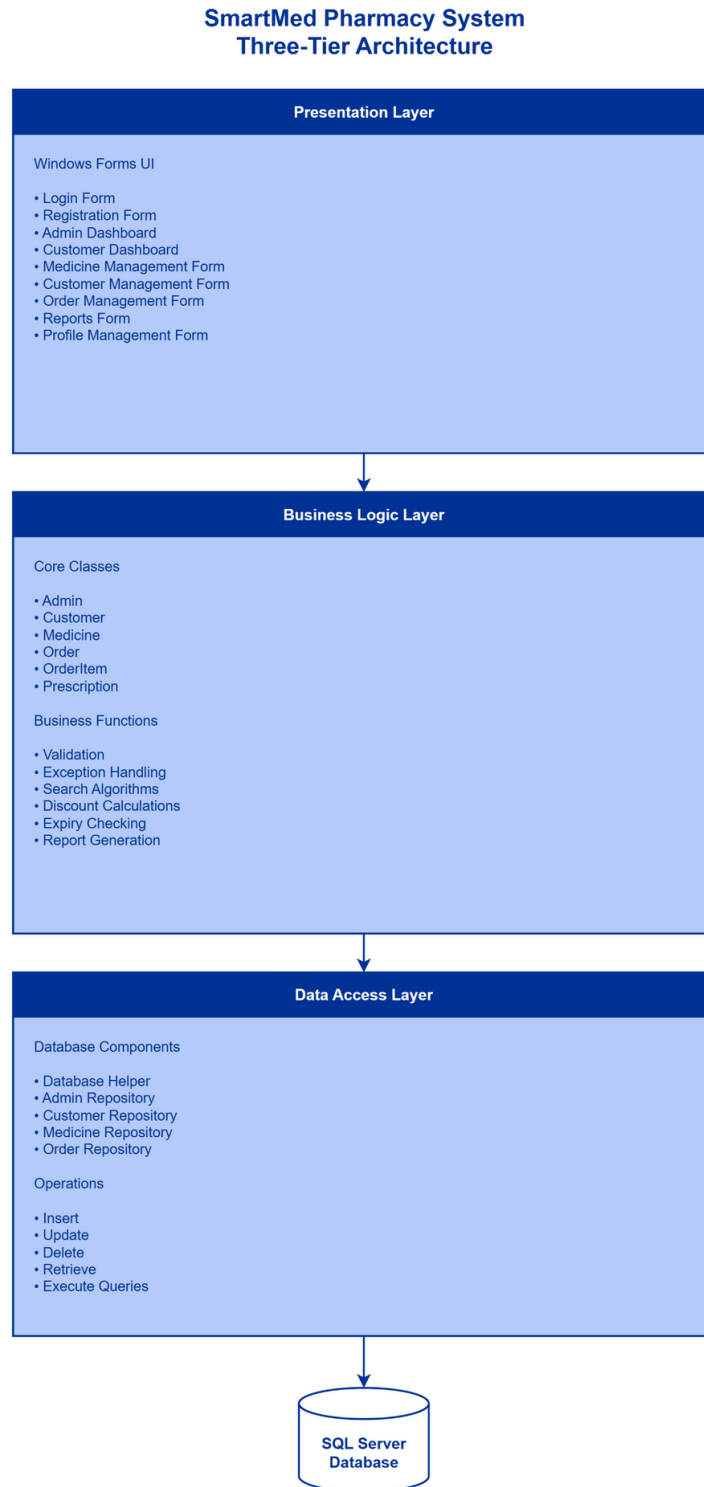
- Microsoft Windows 10 or later.
- .NET Framework 4.8
- Microsoft SQL Server
- Visual Studio 2019 or later with .NET desktop development workload

Development Tools and Technologies

- Programming Language: C#
- User Interface: Windows Forms
- Database: Microsoft SQL Server
- Data Access Technology: ADO.NET
- Integrated Development Environment (IDE): Visual Studio 2022
- Version Control: Git and GitHub
- Diagramming Tool: Draw.io
- Architecture Pattern: Three-Tier Architecture

Design Diagrams

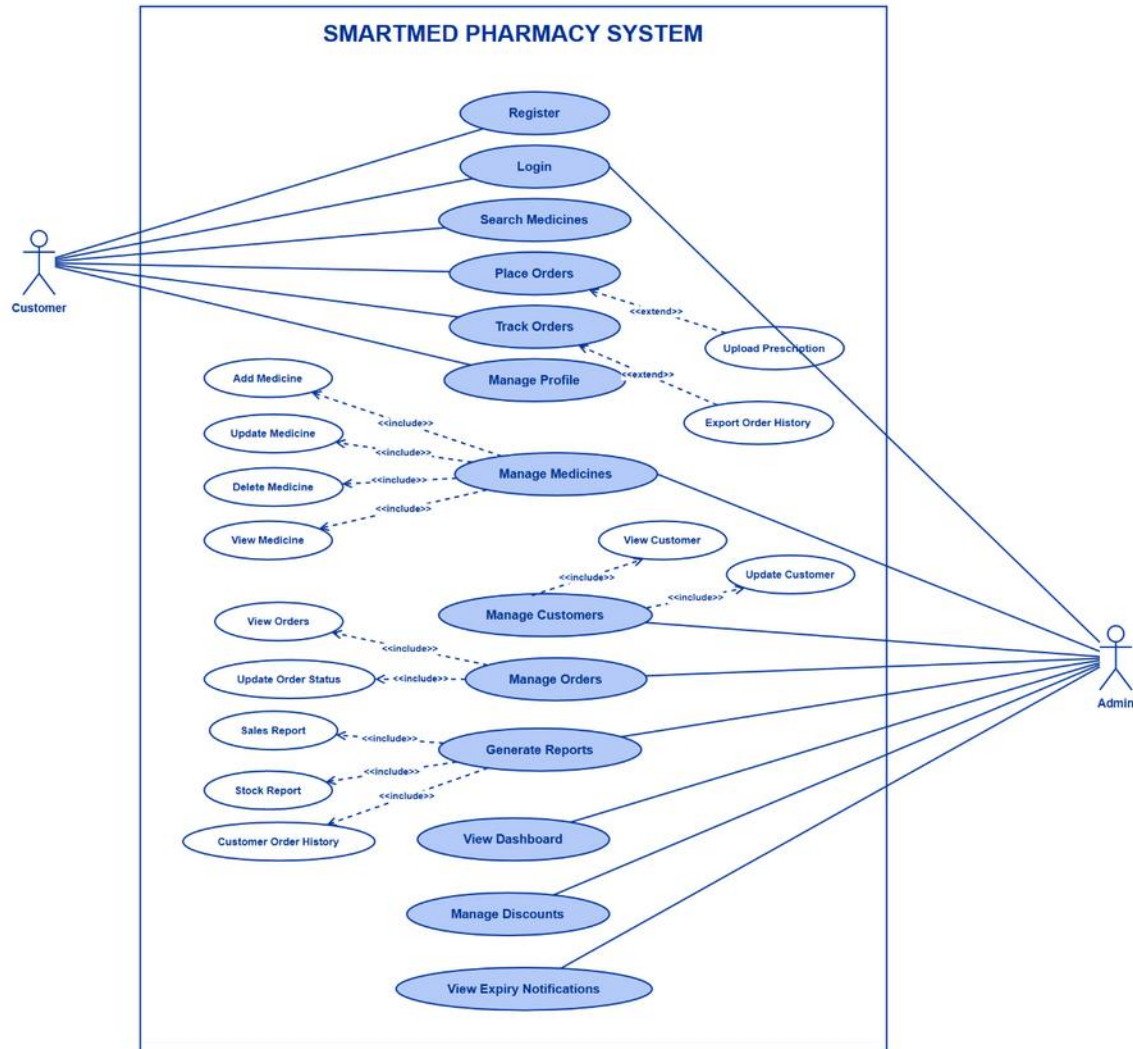
Architecture Diagram



The architecture diagram was drawn at the start of the project to decide how the application should be split into clear parts implementation. A layered structure was chosen so that screens, business rules, and database access do not get mixed together, which makes the system easier to build, test, and explain in a coursework report. This approach is common in desktop business applications and matches what was taught about separation of concerns.

The diagram also justified why SmartMed has separate folders for Forms, Services, Data, and Models. Each layer has one main job: the user interface handles clicks and display, services apply pharmacy rules, repositories talk to SQL Server, and model classes simply carry data between those layers. This general structure supported both the admin and customer sides of the application without duplicating database code on every screen.

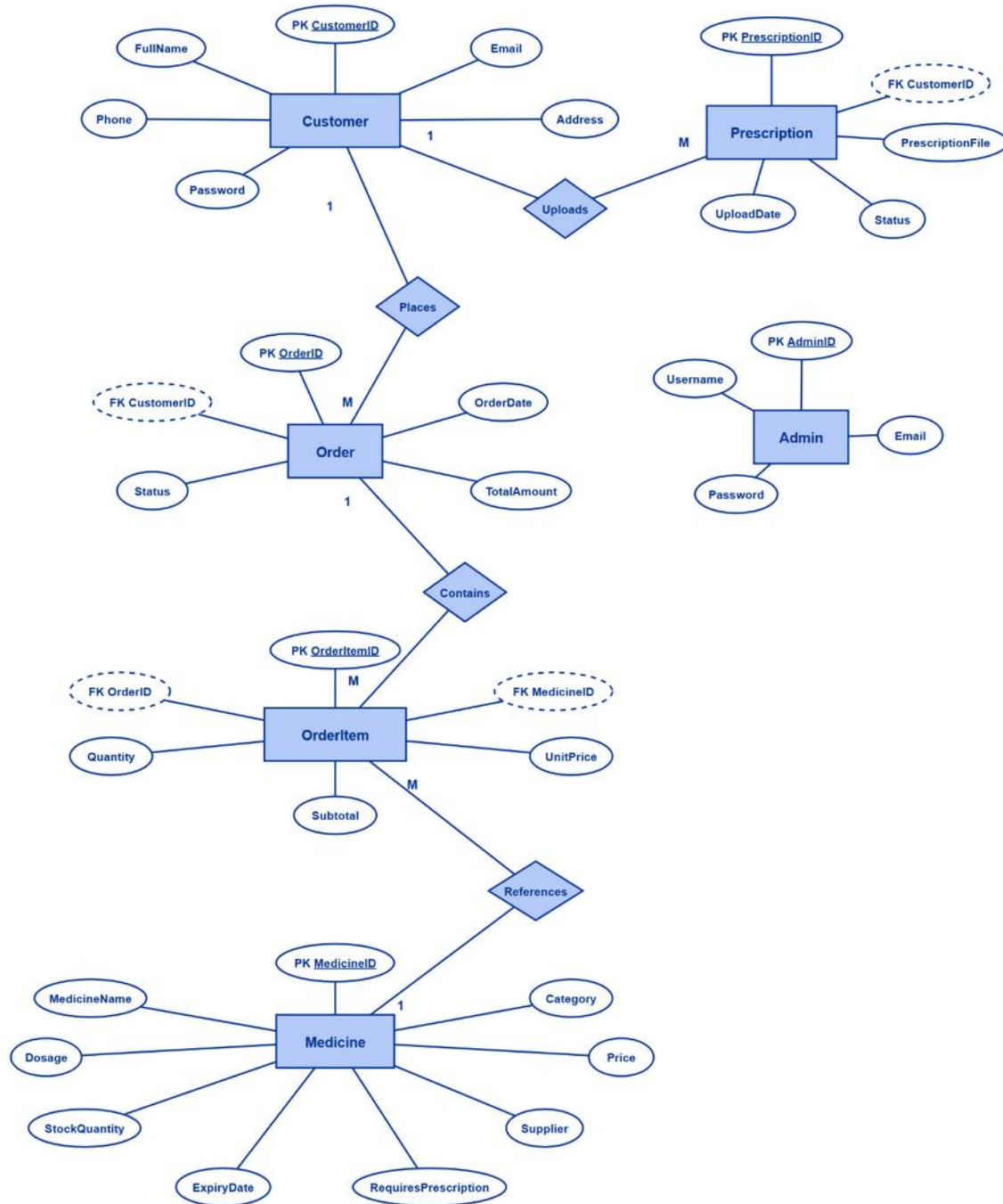
Use Case Diagram



The use case diagram was created during requirements analysis to confirm that both user roles - admin and customer - have a complete set of features before detailed design began. It gives a high-level, non-technical view of what the system should offer to the outside world, without showing buttons, code, or database tables.

Drawing this diagram early helped justify the scope of SmartMed: admin work (stock, customers, orders, reports) and customer work (register, search, cart, track orders) are clearly separated. It also highlighted relationships such as placing an order depending on searching medicines first, which later guided how the customer screens were grouped in the application.

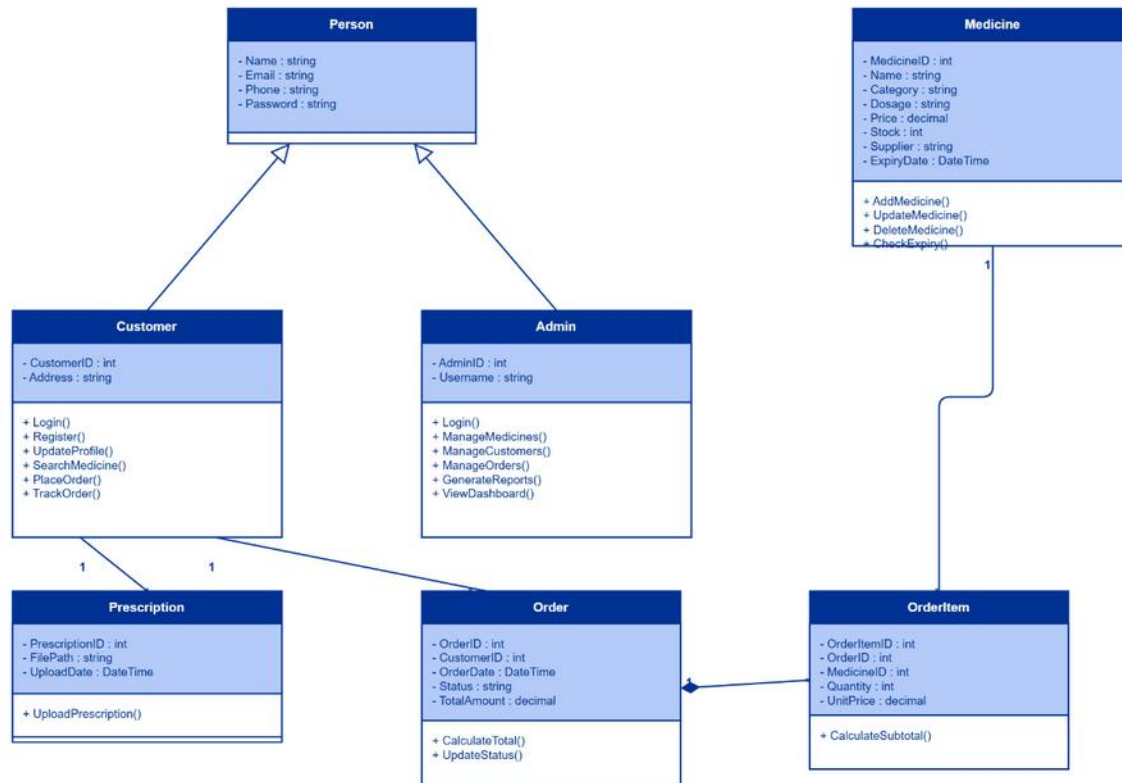
ER Diagram



An entity-relationship diagram was prepared before writing the SQL script so that data could be stored in a logical and consistent way. The main purpose was to plan how pharmacy records link together -for example, customers place orders, and each order contains line items -so that reports and order history would still make sense after many transactions.

This diagram justified the use of primary keys, foreign keys, and status rules in the database. It reduced the risk of designing tables that could not support real pharmacy operations, such as keeping past order details even when a medicine is later removed from the catalogue.

Class Diagram

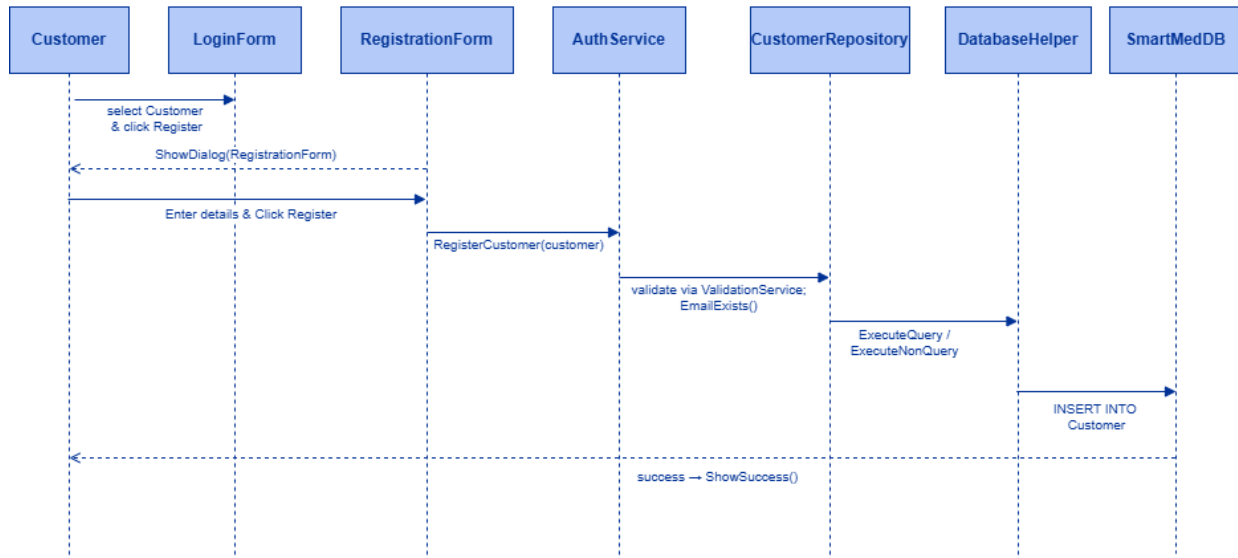


The class diagram was used to plan the object-oriented structure of SmartMed before implementation. It shows how domain types relate to one another through inheritance (for example, Admin and Customer extending a common User type) and how services and repositories support those entities.

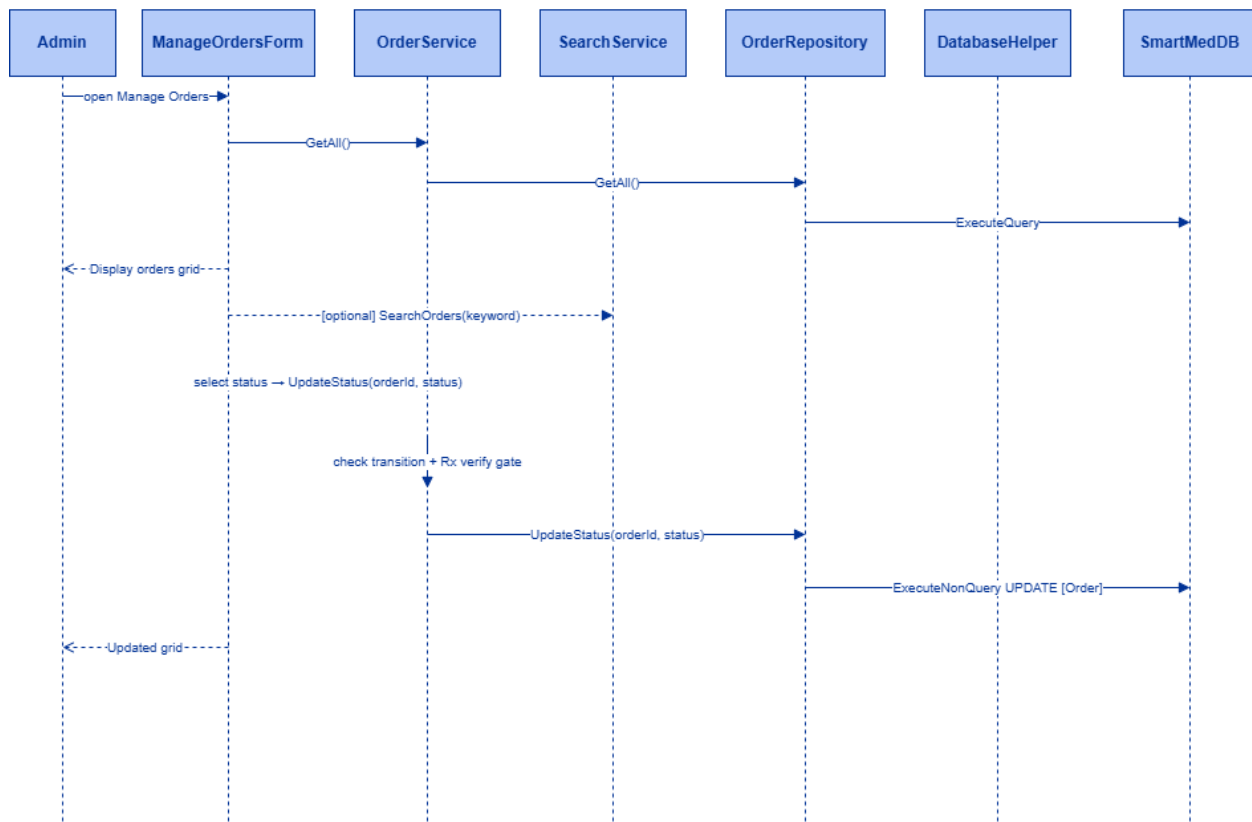
The diagram justified applying OOP concepts such as generalisation, encapsulation, and class responsibility at design time. It acted as a map from the problem domain to C# classes, even though the final code places most behaviour in service classes rather than inside every model class, which is explained further in the class descriptions section.

Sequence Diagram

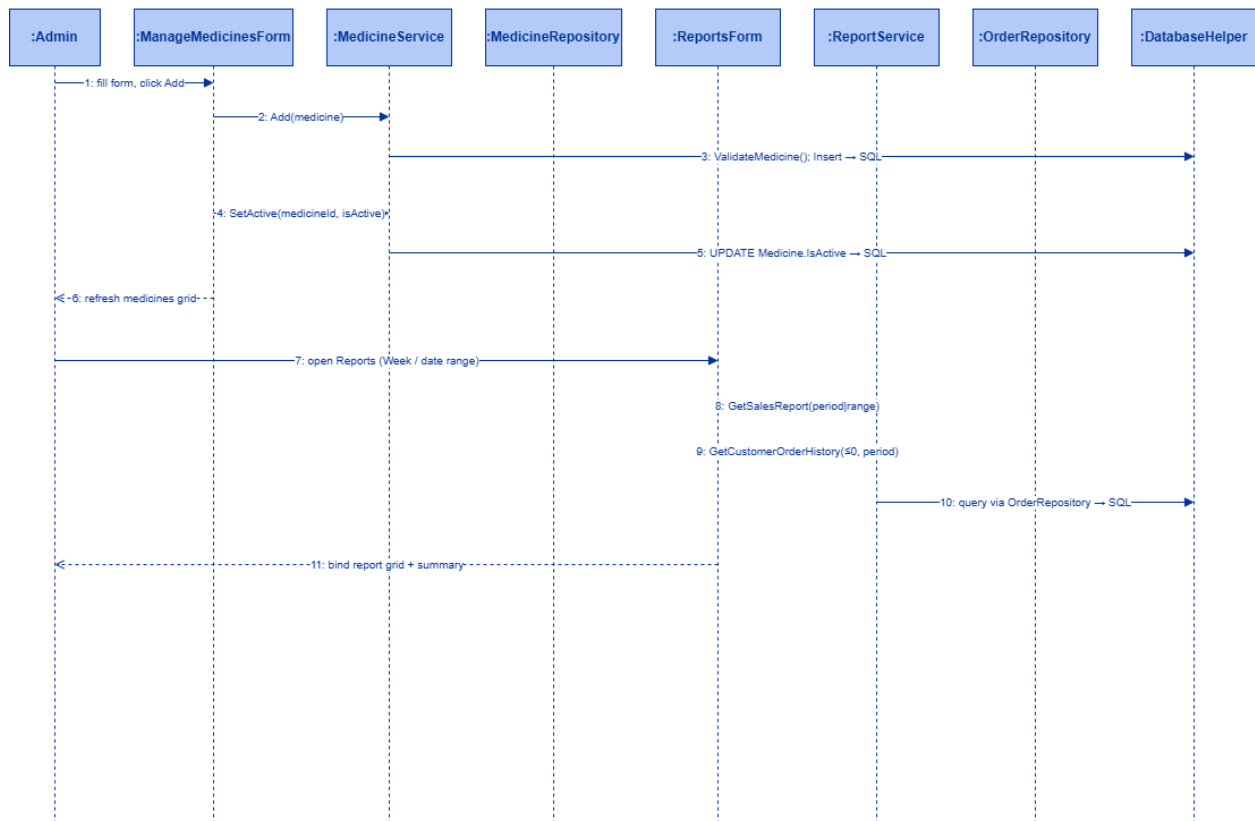
Sequence Diagram 1 — Customer Registration



Sequence Diagram 2 — Admin Manage Orders



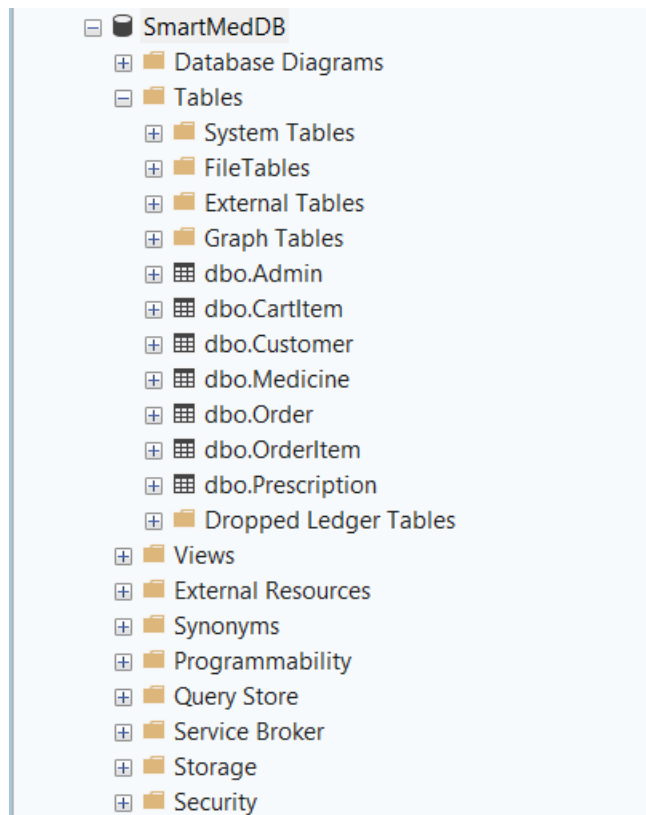
Sequence Diagram 3 — Admin Medicine CRUD & Reports

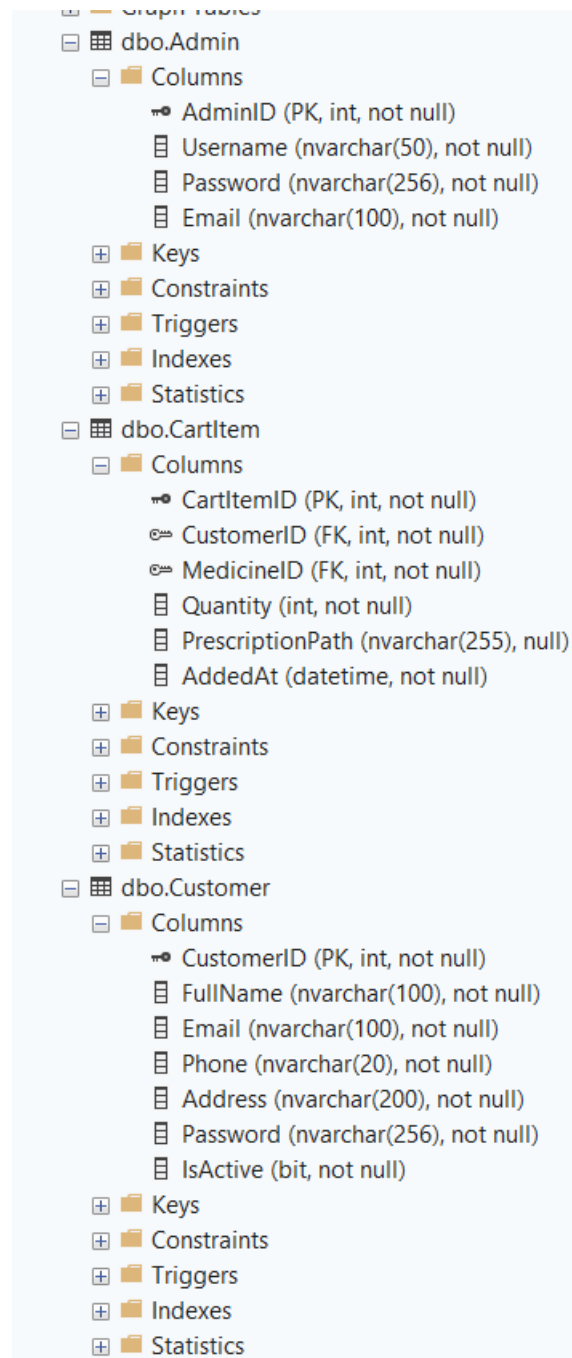


Sequence diagrams were included to show the time order of interactions for important scenarios such as login, placing an order, and managing medicines. They justify how a user action on a form flows through services and repositories before data reaches the database, which is easier to understand as a message sequence than from code alone.

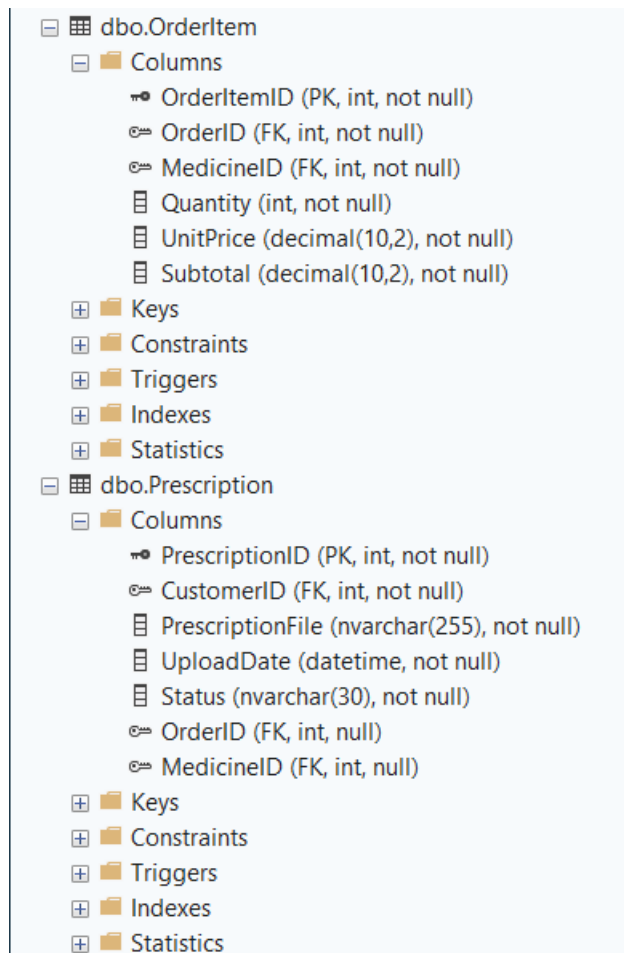
These diagrams were especially useful because some screens call a service layer while the overall rule remains the same: the user interface should not execute SQL directly. Comparing the sequence diagrams with the finished project helped verify that the implementation follows the intended layered flow.

Database Design





[-] [table icon] dbo.Medicine
[-] [folder icon] Columns
[key icon] MedicineID (PK, int, not null)
[icon] MedicineName (nvarchar(100), not null)
[icon] Category (nvarchar(50), not null)
[icon] Dosage (nvarchar(50), not null)
[icon] Price (decimal(10,2), not null)
[icon] StockQuantity (int, not null)
[icon] Supplier (nvarchar(100), not null)
[icon] ExpiryDate (date, not null)
[icon] RequiresPrescription (bit, not null)
[icon] IsOnPromotion (bit, not null)
[icon] DiscountPercent (decimal(5,2), not null)
[icon] PromotionStartDate (date, null)
[icon] PromotionEndDate (date, null)
[icon] Description (nvarchar(500), null)
[icon] ActiveIngredient (nvarchar(200), null)
[icon] UsageInstructions (nvarchar(500), null)
[icon] Warnings (nvarchar(500), null)
[icon] SideEffects (nvarchar(300), null)
[icon] PackSize (nvarchar(100), null)
[icon] IsActive (bit, not null)
[+] [folder icon] Keys
[+] [folder icon] Constraints
[+] [folder icon] Triggers
[+] [folder icon] Indexes
[+] [folder icon] Statistics
[-] [table icon] dbo.Order
[-] [folder icon] Columns
[key icon] OrderID (PK, int, not null)
[foreign key icon] CustomerID (FK, int, not null)
[icon] OrderDate (datetime, not null)
[icon] Status (nvarchar(30), not null)
[icon] TotalAmount (decimal(10,2), not null)
[icon] PaymentMethod (nvarchar(30), not null)
[icon] PaymentStatus (nvarchar(30), not null)
[icon] PaymentReference (nvarchar(50), null)
[icon] CancellationReason (nvarchar(500), null)
[+] [folder icon] Keys
[+] [folder icon] Constraints



The database design section documents how SmartMedDB stores pharmacy data in a reliable way. A single SQL script creates the database, tables, relationships, and sample records so the application can be demonstrated immediately after installation.

A proper database layout was justified because the application must keep customer orders, stock levels, and prescription files consistent over time. Constraints on order status, foreign keys between orders and customers, and soft-delete on medicines (using an active flag) protect historical records while still allowing day-to-day catalogue changes.

Description of Classes, Properties and Methods

User

```
1 namespace SmartMed.Models
2 {
3     public class User
4     {
5         public string Name { get; set; }
6         public string Email { get; set; }
7         public string Phone { get; set; }
8         public string Password { get; set; }
9         public bool IsActive { get; set; } = true;
10    }
11 }
```

The User class is the parent type for people who can sign in to SmartMed. It represents shared account information that both administrators and customers need, such as name, contact details, and login credentials.

In object-oriented terms, this class demonstrates generalisation: common data is defined once and reused by specialised child classes. The models are kept as simple data containers (properties only). Methods such as login or profile update are not placed inside User because those actions involve database access, validation across several fields, and navigation between screens. That behaviour belongs in AuthService and CustomerService, which is a standard layered approach for maintainability and matches the three-tier architecture.

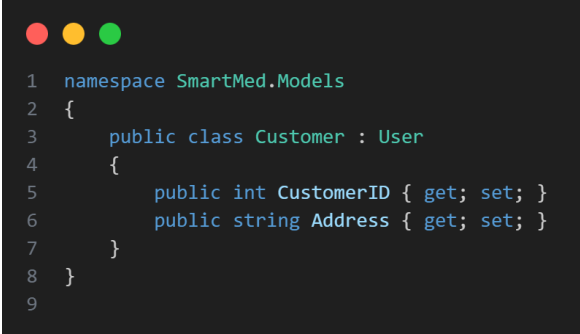
Admin

```
1 namespace SmartMed.Models
2 {
3     public class Admin : User
4     {
5         public int AdminID { get; set; }
6         public string Username { get; set; }
7     }
8 }
```

Admin extends User and represents a pharmacy staff member who manages the system. Inheritance allows Admin to reuse all User fields while adding an administrator identifier and username.

No business methods are declared on Admin itself. Opening the dashboard or managing stock are full application workflows, not single-object operations, so they are implemented in forms and services. After login, an Admin object is stored in the session service so the rest of the admin screens know who is signed in. This is encapsulation at application level: the UI works with a typed object without handling raw database rows.

Customer



```
1 namespace SmartMed.Models
2 {
3     public class Customer : User
4     {
5         public int CustomerID { get; set; }
6         public string Address { get; set; }
7     }
8 }
9
```

Customer extends User and models a registered pharmacy client. Inheritance avoids duplicating name, email, phone, and password on a separate unrelated class.

As with Admin, Customer does not contain methods like PlaceOrder or TrackOrder. Those use cases coordinate the cart, stock, prescriptions, and database transactions. Placing that logic in OrderService and related classes follows the Single Responsibility idea: the Customer class describes who the user is; services describe what the system does for them. The IsActive flag on the user record supports account deactivation without deleting history.

Medicine

```
1  using System;
2
3  namespace SmartMed.Models
4  {
5      public class Medicine
6      {
7          public int MedicineID { get; set; }
8          public string MedicineName { get; set; }
9          public string Category { get; set; }
10         public string Dosage { get; set; }
11         public decimal Price { get; set; }
12         public int StockQuantity { get; set; }
13         public string Supplier { get; set; }
14         public DateTime ExpiryDate { get; set; }
15         public bool RequiresPrescription { get; set; }
16         public decimal DiscountPercent { get; set; }
17         public bool IsOnPromotion { get; set; }
18         public DateTime? PromotionStartDate { get; set; }
19         public DateTime? PromotionEndDate { get; set; }
20         public string Description { get; set; }
21         public string ActiveIngredient { get; set; }
22         public string UsageInstructions { get; set; }
23         public string Warnings { get; set; }
24         public string SideEffects { get; set; }
25         public string PackSize { get; set; }
26         public bool IsActive { get; set; } = true;
27     }
28 }
```

Medicine is a core entity class for products in the pharmacy catalogue. It groups everything the application needs to know about a item for sale -identity, pricing, stock, expiry, promotions, and whether a prescription is required.

The class uses encapsulation through public properties that map to database columns. Behaviour such as checking expiry, applying a promotion price, or deciding if an item can be sold is handled in MedicineService because those rules may combine several fields and must be applied consistently on admin screens, search, cart, and checkout. Keeping Medicine as a data class makes repositories straightforward: they load and save records without embedding pharmacy policy.

Order

```
1 using System;
2
3 namespace SmartMed.Models
4 {
5     public class Order
6     {
7         public int OrderID { get; set; }
8         public int CustomerID { get; set; }
9         public string CustomerName { get; set; }
10        public DateTime OrderDate { get; set; }
11        public string Status { get; set; }
12        public decimal TotalAmount { get; set; }
13        public string PaymentMethod { get; set; }
14        public string PaymentStatus { get; set; }
15        public string PaymentReference { get; set; }
16        public string CancellationReason { get; set; }
17    }
18 }
19
```

Order represents one customer purchase as a whole. It is an association class in the domain model linking a customer to a point in time, a status, and a total amount.

Order does not include methods like CalculateTotal in the model layer because totals are produced when the order is built in OrderService from line items and current prices. Status changes also pass through service and repository layers so rules (for example, when cancellation is allowed) are enforced in one place. This separation keeps the entity easy to bind to grids and reports while preserving business control in services.

OrderItem

```
1 namespace SmartMed.Models
2 {
3     public class OrderItem
4     {
5         public int OrderItemID { get; set; }
6         public int OrderID { get; set; }
7         public int MedicineID { get; set; }
8         public string MedicineName { get; set; }
9         public int Quantity { get; set; }
10        public decimal UnitPrice { get; set; }
11        public decimal Subtotal { get; set; }
12        public decimal ListPrice { get; set; }
13        public decimal DiscountPercent { get; set; }
14        public bool RequiresPrescription { get; set; }
15    }
16 }
```

OrderItem links an order to one medicine line with quantity and price information captured at purchase time. It supports the one-to-many relationship between orders and products in the database design.

Storing unit price and subtotal on each line is an important design choice: if catalogue prices change later, historical orders still show what the customer actually paid. The class remains property-based; calculation and validation happen when OrderService creates items during checkout.

Prescription

```
1 using System;
2
3 namespace SmartMed.Models
4 {
5     public class Prescription
6     {
7         public int PrescriptionID { get; set; }
8         public int CustomerID { get; set; }
9         public int OrderID { get; set; }
10        public int? MedicineID { get; set; }
11        public string PrescriptionFile { get; set; }
12        public DateTime UploadDate { get; set; }
13        public string Status { get; set; }
14    }
15 }
```

Prescription models an uploaded prescription file tied to a customer order, and optionally to a specific medicine when more than one prescription-required product is in the same checkout.

File copying, status workflow (pending, verified, rejected), and admin approval are implemented in PrescriptionService rather than in the entity class, because they involve the file system and cross-record updates. The Prescription class therefore supports encapsulation of data while the service layer encapsulates the verification process.

Service Classes

Service classes group the main business behaviour of SmartMed. They work between forms and repositories.

AuthService handles login, registration, and password changes.

MedicineService manages the catalogue, stock rules, promotions, and expiry checks.

OrderService and CartService handle shopping and checkout.

PrescriptionService manages uploaded prescription files and admin approval.

ReportService prepares dashboard and report figures.

ValidationService and PasswordHasher provide shared technical support.

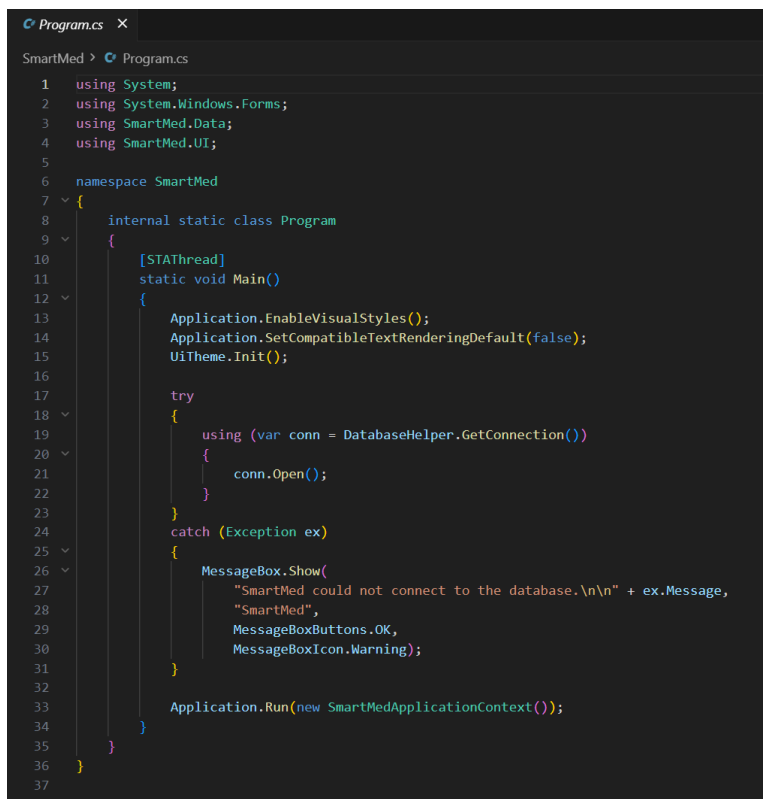
This layout applies object-oriented design at module level: each service has a focused responsibility, collaborates with repositories (composition), and works with model objects passed as parameters. Forms depend on services, not on SQL, which keeps the presentation layer thin and easier to change.

Implementation

Brief Description of Functions

Application startup

When SmartMed starts, Program.Main enables visual styles, initialises the UI theme, and tests the SQL Server connection through DatabaseHelper. If the database is unreachable, a warning appears but the app still opens. Application.Run starts SmartMedApplicationContext, which shows LoginForm first. After login the context swaps to AdminHostForm or CustomerHostForm; on logout it clears Session and CartService and returns to login.



```
1 using System;
2 using System.Windows.Forms;
3 using SmartMed.Data;
4 using SmartMed.UI;
5
6 namespace SmartMed
7 {
8     internal static class Program
9     {
10         [STAThread]
11         static void Main()
12         {
13             Application.EnableVisualStyles();
14             Application.SetCompatibleTextRenderingDefault(false);
15             UITheme.Init();
16
17             try
18             {
19                 using (var conn = DatabaseHelper.GetConnection())
20                 {
21                     conn.Open();
22                 }
23             }
24             catch (Exception ex)
25             {
26                 MessageBox.Show(
27                     "SmartMed could not connect to the database.\n\n" + ex.Message,
28                     "SmartMed",
29                     MessageBoxButtons.OK,
30                     MessageBoxIcon.Warning);
31             }
32
33             Application.Run(new SmartMedApplicationContext());
34         }
35     }
36 }
37
```

```

SmartMedApplicationContext.cs
SmartMed > UI > SmartMedApplicationContext.cs

1  using System;
2  using System.Windows.Forms;
3  using SmartMed.Services;
4
5  namespace SmartMed.UI
6  {
7      public sealed class SmartMedApplicationContext : ApplicationContext
8      {
9          private static SmartMedApplicationContext _current;
10         private bool _handingOffToLogin;
11
12         public static SmartMedApplicationContext Current => _current;
13
14         public SmartMedApplicationContext()
15         {
16             _current = this;
17             ShowLogin(isAfterLogout: false);
18         }
19
20         public void ShowLoginAfterLogout()
21         {
22             CartService.Unload();
23             Session.Clear();
24             _handingOffToLogin = true;
25
26             var previous = MainForm;
27             if (previous != null)
28                 previous.FormClosed -= OnMainFormClosed;
29
30             ShowLogin(isAfterLogout: true, source: previous);
31             previous?.Close();
32             _handingOffToLogin = false;
33         }
34     }

```

```

35     private void ShowLogin(bool isAfterLogout, Form source = null)
36     {
37         var login = new LoginForm();
38         if (isAfterLogout)
39             login.ResetAfterLogout();
40
41         CopyWindowStateAndSize(source, login);
42
43         login.LoginSucceeded += OnLoginSucceeded;
44         login.FormClosed += OnLoginFormClosed;
45         MainForm = login;
46         login.Show();
47     }
48
49     private static void CopyWindowStateAndSize(Form source, Form target)
50     {
51         if (source == null || target == null)
52             return;
53
54         target.StartPosition = FormStartPosition.Manual;
55
56         if (source.WindowState == FormWindowState.Maximized)
57         {
58             var restore = source.RestoreBounds;
59             if (restore.Width > 0 && restore.Height > 0)
60             {
61                 target.Location = restore.Location;
62                 target.Size = restore.Size;
63             }
64             target.WindowState = FormWindowState.Maximized;
65         }
66         else if (source.WindowState == FormWindowState.Normal)
67         {

```

```

65     }
66     else if (source.WindowState == FormWindowState.Normal)
67     {
68         target.WindowState = FormWindowState.Normal;
69         target.Location = source.Location;
70         target.Size = source.Size;
71     }
72 }
73
74 private void OnLoginSucceeded(object sender, EventArgs e)
75 {
76     if (!(sender is LoginForm login))
77         return;
78
79     login.LoginSucceeded -= OnLoginSucceeded;
80     login.FormClosed -= OnLoginFormClosed;
81
82     if (Session.IsAdminLoggedIn)
83     {
84         var adminHost = new AdminHostForm();
85         CopyWindowStateAndSize(login, adminHost);
86         adminHost.FormClosed += OnMainFormClosed;
87         MainForm = adminHost;
88         adminHost.Show();
89         login.Close();
90         return;
91     }
92
93     var customerHost = new CustomerHostForm();
94     CopyWindowStateAndSize(login, customerHost);
95     customerHost.FormClosed += OnMainFormClosed;
96     MainForm = customerHost;
97     customerHost.Show();
98     login.Close();

```

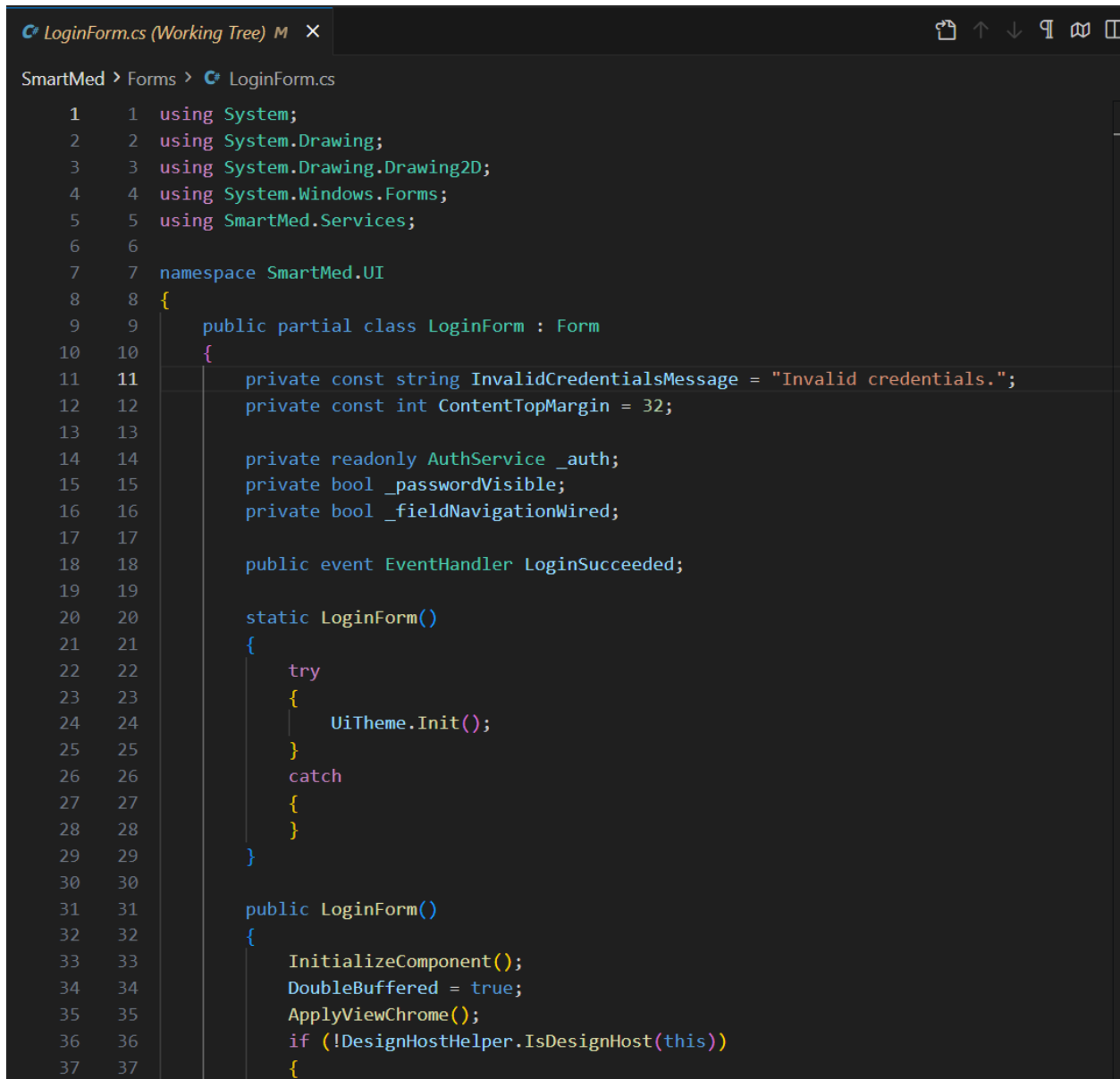
```

99     }
100 }
101
102 private void OnLoginFormClosed(object sender, FormClosedEventArgs e)
103 {
104     var closed = sender as LoginForm;
105     if (closed != null)
106     {
107         closed.FormClosed -= OnLoginFormClosed;
108         if (MainForm == closed)
109             ExitThread();
110     }
111 }
112
113 private void OnMainFormClosed(object sender, FormClosedEventArgs e)
114 {
115     var form = sender as Form;
116     if (form != null)
117         form.FormClosed -= OnMainFormClosed;
118
119     if (!_handingOffToLogin)
120         ExitThread();
121 }
122 }
123

```

Login

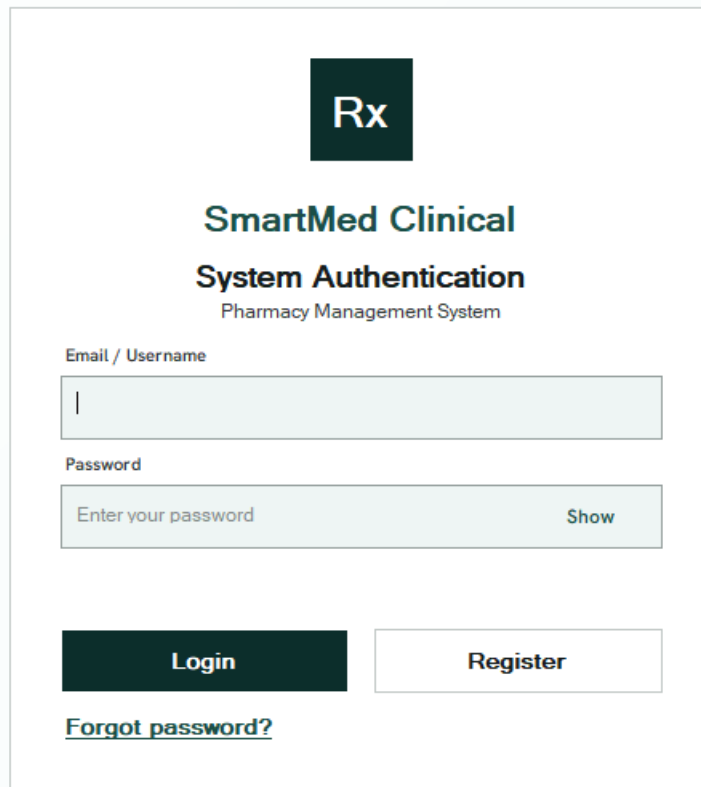
Login is the entry point for administrators and customers. On LoginForm the user enters username or email and password, then clicks Login. PerformLogin uses ValidationService, then AuthService.AdminLogin or CustomerLogin. PasswordHasher verifies the hash. On success Session stores the user, CartService loads the customer cart, and the application context opens the correct home screen. Wrong credentials show a warning on the login card.



```
1 1 using System;
2 2 using System.Drawing;
3 3 using System.Drawing.Drawing2D;
4 4 using System.Windows.Forms;
5 5 using SmartMed.Services;
6 6
7 7 namespace SmartMed.UI
8 8 {
9 9     public partial class LoginForm : Form
10 10 {
11 11     private const string InvalidCredentialsMessage = "Invalid credentials.";
12 12     private const int ContentTopMargin = 32;
13 13
14 14     private readonly AuthService _auth;
15 15     private bool _passwordVisible;
16 16     private bool _fieldNavigationWired;
17 17
18 18     public event EventHandler LoginSucceeded;
19 19
20 20     static LoginForm()
21 21     {
22 22         try
23 23         {
24 24             UiTheme.Init();
25 25         }
26 26         catch
27 27         {
28 28         }
29 29     }
30 30
31 31     public LoginForm()
32 32     {
33 33         InitializeComponent();
34 34         DoubleBuffered = true;
35 35         ApplyViewChrome();
36 36         if (!DesignHostHelper.IsDesignHost(this))
37 37         {
```

```
AuthService.cs x
SmartMed > Services > AuthService.cs

1  using SmartMed.Data;
2  using SmartMed.Models;
3
4  namespace SmartMed.Services
5  {
6      public class AuthService
7      {
8          private readonly AdminRepository _adminRepo = new AdminRepository();
9          private readonly CustomerRepository _customerRepo = new CustomerRepository();
10
11         public Admin AdminLogin(string username, string password)
12         {
13             // Verify that the admin entered a username/email and password.
14             if (ValidationService.IsNullOrEmpty(username) || ValidationService.IsNullOrEmpty(password))
15                 throw new System.ArgumentException("Username and password are required.");
16
17             // Check the credentials against the registered admin account.
18             var admin = _adminRepo.GetByUsernameOrEmail(username.Trim());
19             if (admin == null || !PasswordHasher.Verify(password, admin.Password))
20                 return null;
21
22             UpgradePasswordIfNeeded(
23                 admin.Password,
24                 password,
25                 hash =>
26                 {
27                     _adminRepo.UpdatePassword(admin.AdminID, hash);
28                     admin.Password = hash;
29                 });
30
31             return admin;
32         }
33
34         public Customer CustomerLogin(string email, string password)
35         {
36             // Verify that the customer entered valid login credentials.
37             if (!ValidationService.IsValidEmail(email))
```



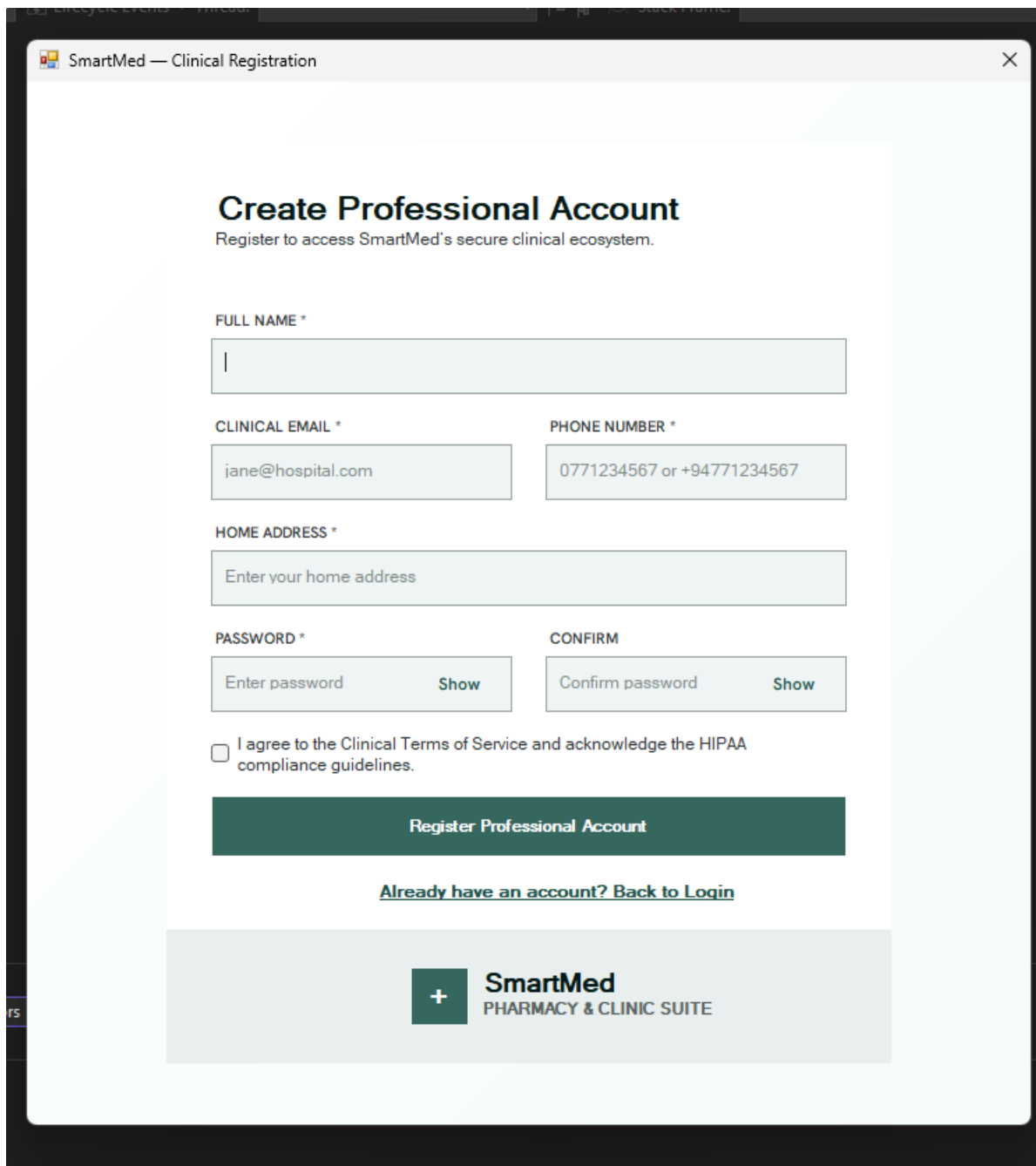
The image shows a login and registration form for 'SmartMed Clinical'. At the top is a dark green square logo with 'Rx' in white. Below it, the text 'SmartMed Clinical' is in bold, followed by 'System Authentication' and 'Pharmacy Management System' in a smaller font. The form has two input fields: 'Email / Username' and 'Password'. The 'Password' field has a 'Show' button next to it. Below the input fields are two buttons: 'Login' (dark green) and 'Register' (light gray). At the bottom, there is a link for '[Forgot password?](#)'.

Customer registration

Registration creates a new customer account. The user opens `RegistrationForm` from login, fills personal details, accepts terms, and clicks Register. The form validates passwords, builds a `Customer` object, and calls `AuthService.RegisterCustomer`, which validates input, hashes the password, and inserts via `CustomerRepository`. Success shows a confirmation panel; duplicate email or weak password shows an error message.

```
RegistrationForm.cs X
SmartMed > Forms > RegistrationForm.cs

1  using System;
2  using System.Drawing;
3  using System.Drawing.Drawing2D;
4  using System.Windows.Forms;
5  using SmartMed.Models;
6  using SmartMed.Services;
7
8  namespace SmartMed.UI
9  {
10     public partial class RegistrationForm : Form
11     {
12         private const int ContentTopMargin = 24;
13
14         private readonly AuthService _auth;
15         private bool _passwordVisible;
16         private bool _confirmVisible;
17         private bool _passwordRemaskWired;
18
19         public RegistrationForm()
20         {
21             InitializeComponent();
22             DoubleBuffered = true;
23             ApplyViewChrome();
24             if (!DesignHostHelper.IsDesignHost(this))
25             {
26                 _auth = new AuthService();
27                 WireRuntimeBehavior();
28             }
29         }
30
31         protected override void OnLoad(EventArgs e)
32         {
33             base.OnLoad(e);
34             ApplyViewChrome();
35         }
36     }
37 }
```



The image shows a web browser window titled "SmartMed — Clinical Registration". The main heading is "Create Professional Account" with a subtext "Register to access SmartMed's secure clinical ecosystem." The form contains several input fields: "FULL NAME *" (a single-line text box), "CLINICAL EMAIL *" (a single-line text box containing "jane@hospital.com"), "PHONE NUMBER *" (a single-line text box containing "0771234567 or +94771234567"), "HOME ADDRESS *" (a single-line text box containing "Enter your home address"), "PASSWORD *" (a single-line text box containing "Enter password" and a "Show" button), and "CONFIRM" (a single-line text box containing "Confirm password" and a "Show" button). Below the password fields is a checkbox labeled "I agree to the Clinical Terms of Service and acknowledge the HIPAA compliance guidelines." At the bottom of the form is a large green button labeled "Register Professional Account". Below the button is a link: "Already have an account? Back to Login". At the very bottom of the form is a logo consisting of a green square with a white plus sign, followed by the text "SmartMed" and "PHARMACY & CLINIC SUITE" below it.

SmartMed — Clinical Registration

Create Professional Account

Register to access SmartMed's secure clinical ecosystem.

FULL NAME *

CLINICAL EMAIL *

PHONE NUMBER *

HOME ADDRESS *

PASSWORD *

CONFIRM

☐ I agree to the Clinical Terms of Service and acknowledge the HIPAA compliance guidelines.

Register Professional Account

[Already have an account? Back to Login](#)

 **SmartMed**
PHARMACY & CLINIC SUITE

Forgot password

Forgot password helps users who cannot sign in. ForgotPasswordForm collects email or username and a new password. BtnSave_Click calls AuthService.ResetPassword to find the account and save a new hash. Success confirms the reset and the user returns to login with the new password.

```
ForgotPasswordForm.cs x
SmartMed > Forms > ForgotPasswordForm.cs

1  using System;
2  using System.Drawing;
3  using System.Drawing.Drawing2D;
4  using System.Windows.Forms;
5  using SmartMed.Services;
6
7  namespace SmartMed.UI
8  {
9      public sealed partial class ForgotPasswordForm : Form
10     {
11         public ForgotPasswordForm()
12         {
13             InitializeComponent();
14             DoubleBuffered = true;
15             ApplyViewChrome();
16             if (!DesignHostHelper.IsDesignHost(this))
17                 WireRuntimeBehavior();
18         }
19
20         protected override void OnLoad(EventArgs e)
21         {
22             base.OnLoad(e);
23             ApplyViewChrome();
24         }
25
26         private void ApplyViewChrome()
27         {
28             AuthFormView.ApplyCardBorder(panelCard);
29             AuthFormView.StyleFieldLabels(lblIdentity, lblCurrent, lblNew, lblConfirm);
30             AuthFormView.ApplySoftFieldSurfaces(txtIdentity, txtCurrent, txtNew, txtConfirm);
31             UiTheme.ApplyFlatButton(btnSave, UiButtonStyle.Primary);
32             UiTheme.ApplyFlatButton(btnCancel, UiButtonStyle.Secondary);
33         }
34
35         private void WireRuntimeBehavior()
```

```
public void ResetPassword(string identity, string currentPassword, string newPassword)
{
    // Require the current password so email/username alone cannot reset an account.
    if (ValidationService.IsNullOrEmpty(identity))
        throw new System.ArgumentException("Email or username is required.");
    if (ValidationService.IsNullOrEmpty(currentPassword))
        throw new System.ArgumentException("Current password is required.");
    if (ValidationService.IsNullOrEmpty(newPassword))
        throw new System.ArgumentException("New password is required.");
    if (newPassword.Length < 6)
        throw new System.ArgumentException("New password must be at least 6 characters.");

    identity = identity.Trim();

    // Try to match an admin first, then a customer by email.
    var admin = _adminRepo.GetByUsernameOrEmail(identity);
    if (admin != null)
    {
        if (!PasswordHasher.Verify(currentPassword, admin.Password))
            throw new System.InvalidOperationException("Current password is incorrect.");
        _adminRepo.UpdatePassword(admin.AdminID, PasswordHasher.Hash(newPassword));
        return;
    }

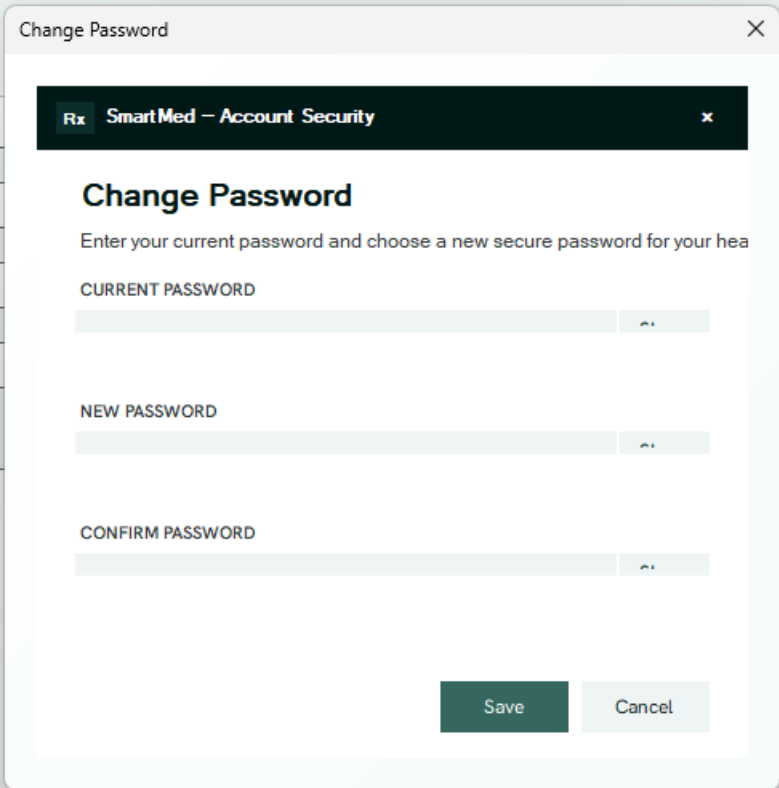
    if (ValidationService.IsValidEmail(identity))
    {
        var customer = _customerRepo.GetByEmail(identity);
        if (customer != null)
```

Change password

Change password updates credentials for a logged-in admin or customer. ChangePasswordForm collects current and new passwords. AuthService.ChangeAdminPassword or CustomerService.ChangePassword verifies the current hash and stores a new one. Customers open it from ProfileManagementForm; admins from the admin shell. Session password fields are cleared after save.

```
ChangePasswordForm.cs X
SmartMed > Forms > ChangePasswordForm.cs

1  using System;
2  using System.Drawing;
3  using System.Drawing.Drawing2D;
4  using System.Windows.Forms;
5  using SmartMed.Services;
6
7  namespace SmartMed.UI
8  {
9      public sealed partial class ChangePasswordForm : Form
10     {
11         private readonly bool _isAdmin;
12
13         public ChangePasswordForm() : this(isAdmin: true)
14         {
15         }
16
17         public ChangePasswordForm(bool isAdmin)
18         {
19             _isAdmin = isAdmin;
20             InitializeComponent();
21             DoubleBuffered = true;
22             ApplyViewChrome();
23             if (!DesignHostHelper.IsDesignHost(this))
24                 WireRuntimeBehavior();
25         }
26
27         protected override void OnLoad(EventArgs e)
28         {
29             base.OnLoad(e);
30             ApplyViewChrome();
31         }
32
33         private void ApplyViewChrome()
34         {
35             AuthFormView.ApplyCardBorder(panelCard);
36             AuthFormView.StyleFieldLabels(lblCurrent, lblNew, lblConfirm);
37             AuthFormView.ApplySoftFieldSurfaces(txtCurrent, txtNew, txtConfirm);
38             UiTheme.ApplyFlatButton(btnSave, UiButtonStyle.Primary);
39         }
40     }
41 }
```

A screenshot of a 'Change Password' dialog box. The dialog has a title bar 'Change Password' with a close button. Inside, there's a dark header bar with 'Rx SmartMed — Account Security' and a close button. The main content area is white and contains the title 'Change Password' and a subtitle 'Enter your current password and choose a new secure password for your head'. Below this are three input fields: 'CURRENT PASSWORD', 'NEW PASSWORD', and 'CONFIRM PASSWORD'. Each field has a toggle icon to its right. At the bottom right are 'Save' and 'Cancel' buttons.

Change Password

Rx SmartMed — Account Security

Change Password

Enter your current password and choose a new secure password for your head

CURRENT PASSWORD

NEW PASSWORD

CONFIRM PASSWORD

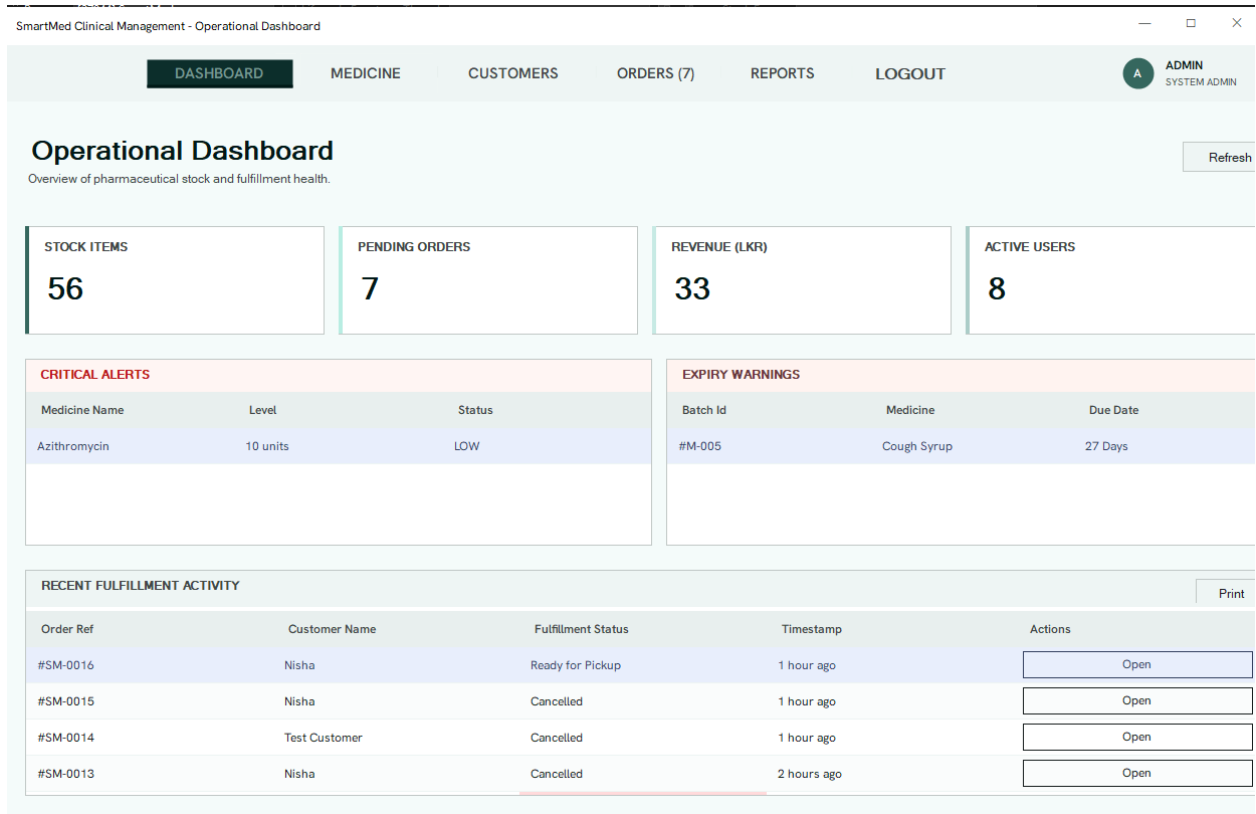
Save Cancel

Admin dashboard

The admin dashboard (Overview) summarises pharmacy operations. `LoadDashboardData` uses `MedicineService` for stock, low-stock and expiry alerts, `OrderService` for pending orders and recent activity, and `ReportService` for sales and customer totals. Stat cards and grids refresh on load or when Refresh is clicked. This covers low-stock alerts and expiry warnings required by the specification.

```
AdminDashboardForm.cs X
SmartMed > Forms > Admin > AdminDashboardForm.cs

1  using System;
2  using System.Collections.Generic;
3  using System.Drawing;
4  using System.Linq;
5  using System.Windows.Forms;
6  using SmartMed.Services;
7
8  namespace SmartMed.UI
9  {
10     public sealed partial class AdminDashboardForm : EmbeddedPageForm
11     {
12         private readonly ReportService _reports;
13         private readonly OrderService _orders;
14         private readonly MedicineService _medicines;
15         private readonly bool _servicesReady;
16
17         private List<object> _recentRows = new List<object>();
18         private bool _chromeApplied;
19
20         public AdminDashboardForm()
21         {
22             InitializeComponent();
23             if (!IsDesignHost())
24             {
25                 _reports = new ReportService();
26                 _orders = new OrderService();
27                 _medicines = new MedicineService();
28                 _servicesReady = true;
29             }
30         }
31
32         protected override bool PreferDesignTimePreview() => !_servicesReady || IsDesignHost();
33
34         public void RefreshData() => RefreshPage();
35
36         protected override void OnLoad(EventArgs e)
37         {
```



Manage medicines

ManageMedicinesForm maintains the catalogue. The admin searches and filters the grid, adds or edits medicines in a dialog (name, category, dosage, price, stock, supplier, expiry, Rx flag, discount, promotions), and saves through MedicineService.Add or Update. Deactivate uses MedicineService.Delete (soft-delete). Row colours highlight expired and low-stock items. Promotions set here affect customer browse and checkout prices.

```
ManageCustomersForm.cs X
SmartMed > Forms > Admin > ManageCustomersForm.cs

1  using System;
2  using System.Collections.Generic;
3  using System.Drawing;
4  using System.Linq;
5  using System.Windows.Forms;
6  using SmartMed.Models;
7  using SmartMed.Services;
8
9  namespace SmartMed.UI
10 {
11     public sealed partial class ManageCustomersForm : EmbeddedPageForm
12     {
13         private const int ActiveOrderDays = 90;
14
15         private readonly CustomerService _customers;
16         private readonly OrderService _orders;
17         private readonly bool _servicesReady;
18
19         private List<CustomerRow> _allRows = new List<CustomerRow>();
20         private List<CustomerRow> _filteredRows = new List<CustomerRow>();
21         private int? _selectedId;
22         private bool _chromeApplied;
23
24         public ManageCustomersForm()
25         {
26             InitializeComponent();
27             if (!IsDesignHost())
28             {
29                 _customers = new CustomerService();
30                 _orders = new OrderService();
31                 _servicesReady = true;
32             }
33         }
34
35         protected override bool PreferDesignTimePreview() => !_servicesReady || IsDesignHost();
36
37         protected override void OnLoad(EventArgs e)
38         {
```

SmartMed Clinical Management - Manage Medicines

DASHBOARD

MEDICINE

CUSTOMERS

ORDERS (7)

REPORTS

LOGOUT

ADMIN
SYSTEM ADMIN

Manage Medicines

Export PDF

+ Add Medicine

Alerts (1)

TOTAL ITEMS
56

LOW STOCK
1

COMPLIANCE
100.0%

Medicine Inventory

Category: All categories Search Search Price: Min Max Clear

ID	Name	Category	Dosage	Stock	Price	Expiry	Rx	Discou	Start Date	End Date	Promc	Status	Edit	Catalog	Delete
#M-00...	Amoxicillin	Antibiotic	250mg	78	12.00	2026-12-...	Rx	—	—	—	No	In Stock	Edit	Deactivate	Delete
#M-00...	Antacid Gel	Gastroint...	200ml	74	8.60	2026-10-...	—	—	—	—	No	In Stock	Edit	Deactivate	Delete
#M-00...	Aspirin	Pain Relief	300mg...	180	6.20	2027-09-...	—	—	—	—	No	In Stock	Edit	Deactivate	Delete
#M-00...	Atorvastatin	Cardiovas...	20mg t...	70	17.96	2027-10-...	Rx	5%	—	—	No	In Stock	Edit	Deactivate	Delete
#M-00...	Azithromycin	Antibiotic	500mg...	10	22.00	2026-11-...	Rx	—	—	—	No	Low Stock	Edit	Deactivate	Delete
#M-00...	Betamethason...	Corticost...	15g tube	42	11.80	2027-07-...	Rx	—	—	—	No	In Stock	Edit	Deactivate	Delete
#M-00...	Bisoprolol	Cardiovas...	5mg ta...	78	13.40	2027-12-...	Rx	—	—	—	No	In Stock	Edit	Deactivate	Delete
#M-00...	Budesonide In...	Respiratory	200mc...	35	29.50	2027-09-...	Rx	—	—	—	No	In Stock	Edit	Deactivate	Delete
#M-00...	Calcium + D3	Supleme...	tablet	130	10.82	2028-05-...	—	12%	2026-07...	2026-08...	Active	In Stock	Edit	Deactivate	Delete
#M-00...	Cetirizine	Antihista...	10mg	120	6.50	2027-12-...	—	—	—	—	No	In Stock	Edit	Deactivate	Delete

System Status: Healthy

Cloud Sync Active

Local Time: 23:2

Manage customers

ManageCustomersForm lets the admin maintain customer accounts. LoadCustomers fills the grid from CustomerService with search and stat tiles. Add/Edit updates contact details;

Activate/Deactivate controls login access without deleting order history. Export and print support admin records from the same screen.

```
ManageCustomersForm.cs X
SmartMed > Forms > Admin > ManageCustomersForm.cs

1  using System;
2  using System.Collections.Generic;
3  using System.Drawing;
4  using System.Linq;
5  using System.Windows.Forms;
6  using SmartMed.Models;
7  using SmartMed.Services;
8
9  namespace SmartMed.UI
10 {
11     public sealed partial class ManageCustomersForm : EmbeddedPageForm
12     {
13         private const int ActiveOrderDays = 90;
14
15         private readonly CustomerService _customers;
16         private readonly OrderService _orders;
17         private readonly bool _servicesReady;
18
19         private List<CustomerRow> _allRows = new List<CustomerRow>();
20         private List<CustomerRow> _filteredRows = new List<CustomerRow>();
21         private int? _selectedId;
22         private bool _chromeApplied;
23
24         public ManageCustomersForm()
25         {
26             InitializeComponent();
27             if (!IsDesignHost())
28             {
29                 _customers = new CustomerService();
30                 _orders = new OrderService();
31                 _servicesReady = true;
32             }
33         }
34
35         protected override bool PreferDesignTimePreview() => !_servicesReady || IsDesignHost();
36
37         protected override void OnLoad(EventArgs e)
```

SmartMed Clinical Management - Manage Customers

DASHBOARD MEDICINE CUSTOMERS ORDERS (7) REPORTS LOGOUT ADMIN SYSTEM ADMIN

Manage Customers

View customer records. Order Activity shows recent ordering; use Account to enable or disable login.

Export Print

TOTAL CUSTOMERS
8

ENABLED
8

DISABLED
0

+ Add Customer Reload Search:

Customer ID	Name	Contact	Last Order	Orders	Order Activity	Edit	Account	Remove
#SM-00009	Test Customer	test@gmail.com / 07564...	2026-07-14	1	ACTIVE	Edit	Deactivate	Remove
#SM-00008	Jane	john2@email.com / 072...	—	0	INACTIVE	Edit	Deactivate	Remove
#SM-00006	Ruwani Fernando	ruwan.fernando@gmail...	—	0	INACTIVE	Edit	Deactivate	Remove
#SM-00005	Kamala Silva	kamala.silva@gmail.com...	2026-07-08	1	ACTIVE	Edit	Deactivate	Remove
#SM-00004	Nimal Perera	nimal.perera@gmail.co...	2026-07-06	1	ACTIVE	Edit	Deactivate	Remove
#SM-00003	Nisha	customer@gmail.com / ...	2026-07-14	8	ACTIVE	Edit	Deactivate	Remove

8 customers

System Status: Healthy Cloud Sync: Active Local Time: 23/2

Manage orders

ManageOrdersForm processes customer orders. The admin filters by status and date, changes status via grid combo (Pending → Ready for Pickup → Delivered) through OrderService.UpdateStatus, and verifies or rejects prescriptions via OrderService.VerifyPrescription / RejectPrescription after reviewing files in PrescriptionFilesDialog.

```
ManageOrdersForm.cs X
SmartMed > Forms > Admin > ManageOrdersForm.cs

1  using System;
2  using System.Collections.Generic;
3  using System.Drawing;
4  using System.Linq;
5  using System.Windows.Forms;
6  using SmartMed.Models;
7  using SmartMed.Services;
8
9  namespace SmartMed.UI
10 {
11     public sealed partial class ManageOrdersForm : EmbeddedPageForm
12     {
13         private const int PageSize = 10;
14
15         private readonly OrderService _orders;
16         private readonly bool _servicesReady;
17         private bool _runtimeWired;
18         private bool _chromeApplied;
19
20         private List<Order> _allOrders = new List<Order>();
21         private List<OrderRow> _filteredRows = new List<OrderRow>();
22         private int _currentPage = 1;
23         private int? _selectedOrderId;
24         private bool _suppressGridEvents;
25         private string _statusEditOriginal;
26
27         public ManageOrdersForm()
28         {
29             InitializeComponent();
30             if (!IsDesignHost())
31             {
32                 _orders = new OrderService();
33                 _servicesReady = true;
34             }
35         }
36
37         protected override bool PreferDesignTimePreview() => !_servicesReady || IsDesignHost();
38     }
39 }
```

SmartMed Clinical Management - Manage Orders

DASHBOARD

MEDICINE

CUSTOMERS

ORDERS (7)

REPORTS

LOGOUT

A ADMIN
SYSTEM ADMIN

Manage Orders

Monitor and process pharmaceutical orders across all clinical departments.

VOLUME
14

AVG TIME
239.4h

FLAGS
00

Filter Options

Search Orders:

Status Category:

Service Date Range:
☐ Use date range to

Apply Filters

Export Data

Order Ref	Customer	Order Date	Total Amount	Prescription	Rx Status	Status	Cancel Reason	Verify	Reject	Cancel	View
#ORD-0016	Nisha (PAT-003-...	Jul 14, 2026 22...	LKR 12.00	3_2_2026071422...	Verified	Ready for Pl...	—	Verify	Reject	Cancel	View
#ORD-0015	Nisha (PAT-003-...	Jul 14, 2026 22...	LKR 8.60	—	—	Pending	No Need	Verify	Reject	Cancel	View
#ORD-0014	Test Customer (...)	Jul 14, 2026 21...	LKR 12.00	9_2_2026071421...	Pending	Pending	Test	Verify	Reject	Cancel	View
#ORD-0013	Nisha (PAT-003-...	Jul 14, 2026 20...	LKR 22.00	3_18_202607142...	Pending	Pending	No stock	Verify	Reject	Cancel	View
#ORD-0012	Nisha (PAT-003-...	Jul 14, 2026 20...	LKR 9.00	—	—	Pending	—	Verify	Reject	Cancel	View

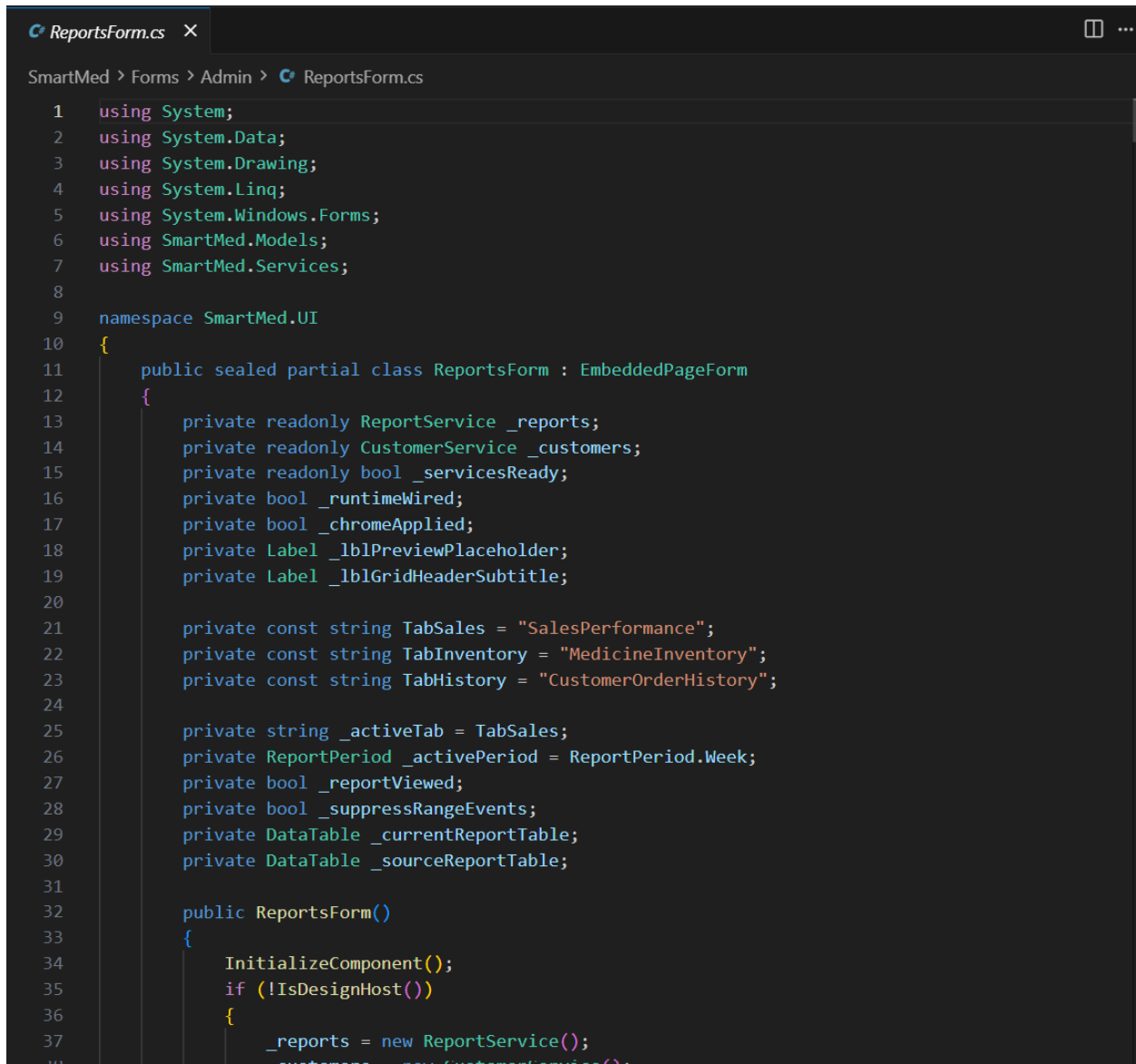
<

System Status: Healthy Cloud Sync Active Local Time: 23:2

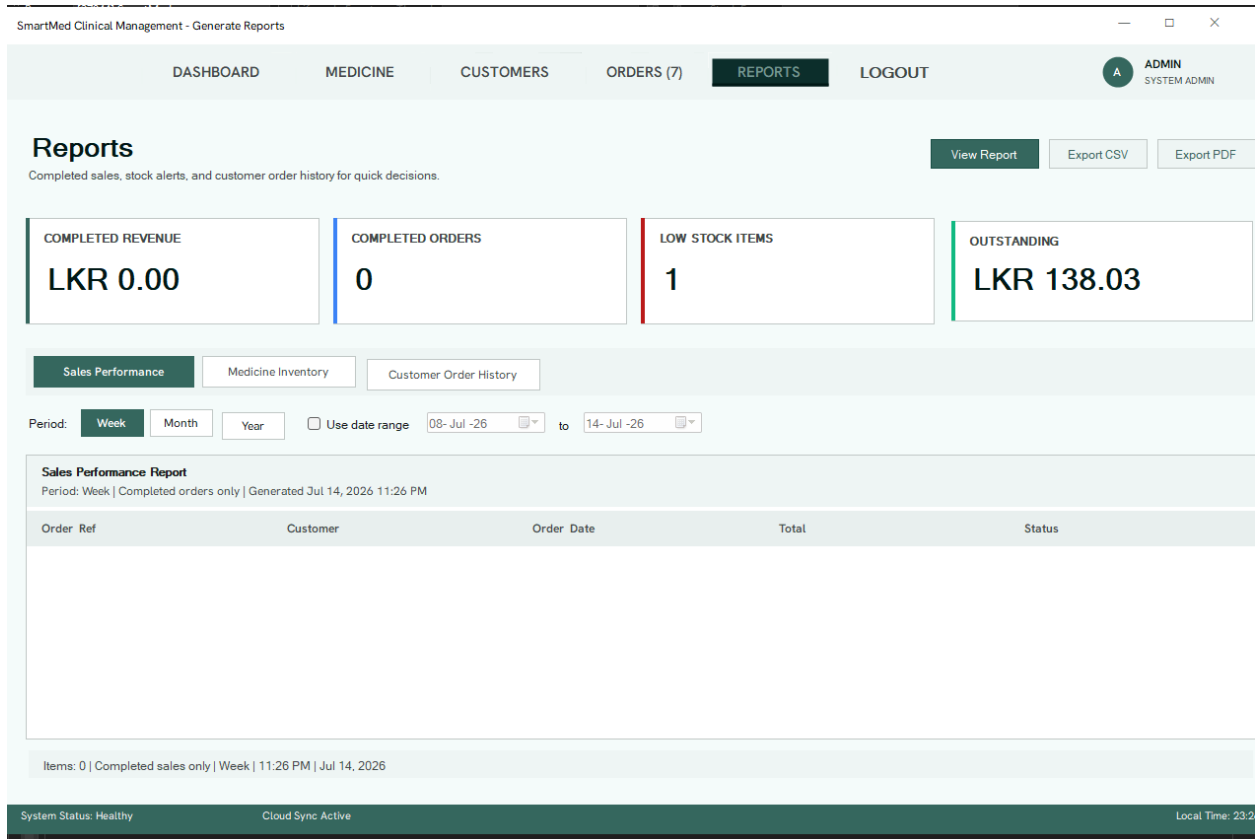
Generate reports

ReportsForm provides sales, inventory, and customer order history tabs. The admin selects a period and optional customer filter, views the preview grid via ReportService, then exports CSV or PDF. This supports operational monitoring without manual SQL.

45



```
1  using System;
2  using System.Data;
3  using System.Drawing;
4  using System.Linq;
5  using System.Windows.Forms;
6  using SmartMed.Models;
7  using SmartMed.Services;
8
9  namespace SmartMed.UI
10 {
11     public sealed partial class ReportsForm : EmbeddedPageForm
12     {
13         private readonly ReportService _reports;
14         private readonly CustomerService _customers;
15         private readonly bool _servicesReady;
16         private bool _runtimeWired;
17         private bool _chromeApplied;
18         private Label _lblPreviewPlaceholder;
19         private Label _lblGridHeaderSubtitle;
20
21         private const string TabSales = "SalesPerformance";
22         private const string TabInventory = "MedicineInventory";
23         private const string TabHistory = "CustomerOrderHistory";
24
25         private string _activeTab = TabSales;
26         private ReportPeriod _activePeriod = ReportPeriod.Week;
27         private bool _reportViewed;
28         private bool _suppressRangeEvents;
29         private DataTable _currentReportTable;
30         private DataTable _sourceReportTable;
31
32         public ReportsForm()
33         {
34             InitializeComponent();
35             if (!IsDesignHost())
36             {
37                 _reports = new ReportService();
38                 _customers = new CustomerService();
39             }
40         }
41     }
42 }
```



Customer home dashboard

CustomerDashboardForm is the customer Home page. DoRefreshPage shows a welcome message, cart item count, open orders, active promotion count, and a recent-orders grid using CartService, OrderService, and MedicineService.

```
1  using System;
2  using System.Drawing;
3  using System.Linq;
4  using System.Windows.Forms;
5  using SmartMed.Services;
6
7  namespace SmartMed.UI
8  {
9      public sealed partial class CustomerDashboardForm : EmbeddedPageForm
10     {
11         private readonly OrderService _orders;
12         private readonly MedicineService _medicines;
13         private readonly bool _servicesReady;
14         private bool _chromeApplied;
15
16         public CustomerDashboardForm()
17         {
18             InitializeComponent();
19             if (!IsDesignHost())
20             {
21                 _orders = new OrderService();
22                 _medicines = new MedicineService();
23                 _servicesReady = true;
24             }
25         }
26
27         protected override bool PreferDesignTimePreview() => !_servicesReady || IsDesignHost();
28
29         protected override void OnLoad(EventArgs e)
30         {
31             base.OnLoad(e);
32             ApplyViewChrome();
33         }
34
35         protected override void DoRefreshPage()
36         {
37             if (!_servicesReady) return;
```

SmartMed Health Portal - Home

HOME BROWSE MEDICINE MY CART MY ORDERS MY PROFILE LOGOUT CONTACT

Customer Home

Welcome, Nisha – browse medicines, manage your cart, and track orders.

ITEMS IN CART
0

ACTIVE ORDERS
8

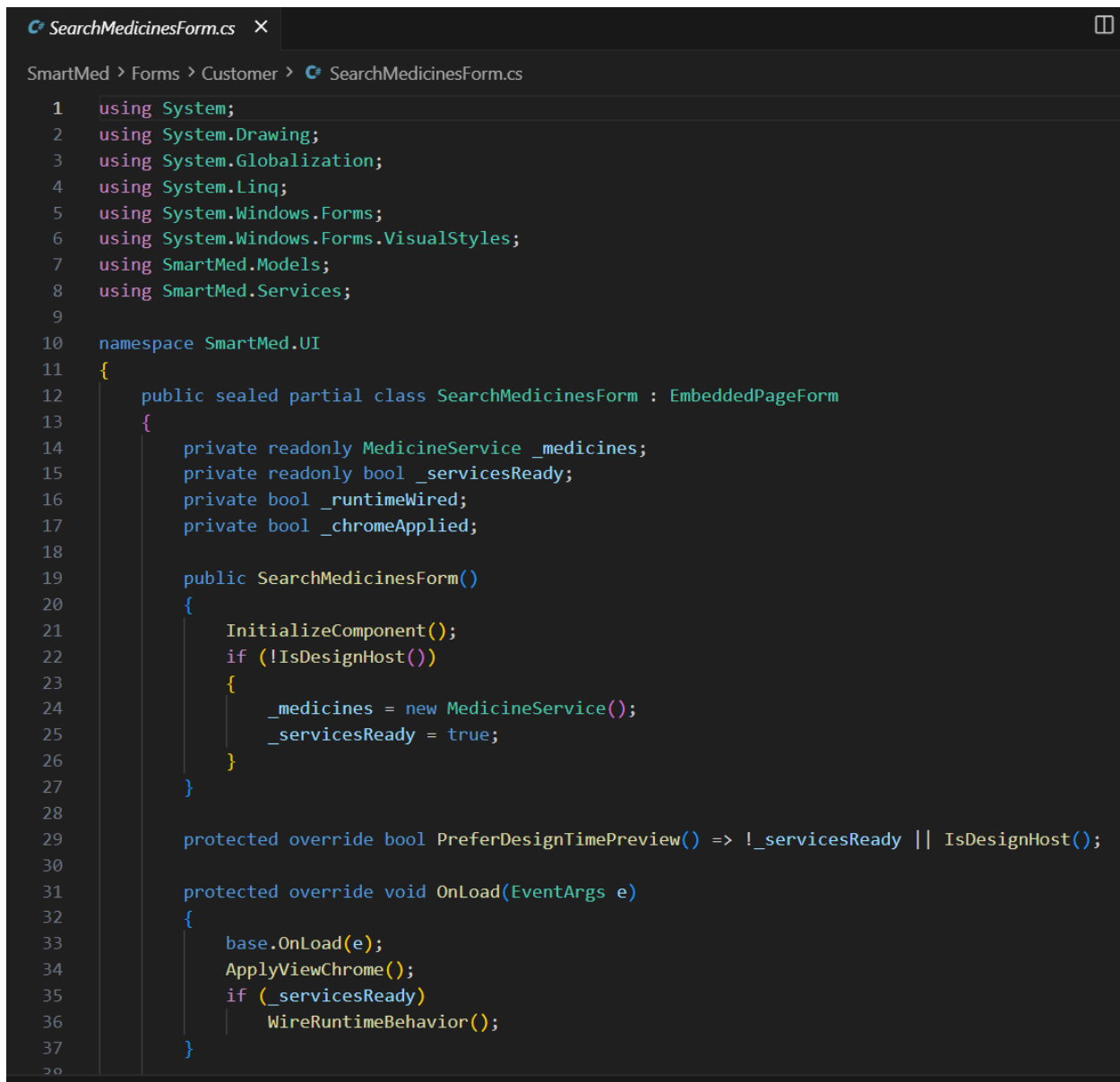
PROMOTIONS
4

Recent Orders			
Order Ref	Order Date	Status	Total
#SM-0016	14-Jul-26 10:15 PM	Ready for Pickup	LKR 12.00
#SM-0015	14-Jul-26 10:13 PM	Cancelled	LKR 8.60
#SM-0013	14-Jul-26 8:59 PM	Cancelled	LKR 22.00
#SM-0012	14-Jul-26 8:59 PM	Pending	LKR 9.00
#SM-0011	14-Jul-26 8:48 PM	Pending	LKR 10.00

Welcome to SmartMed Secure Session Logout Order: 2367

Search medicines and add to cart

SearchMedicinesForm is the customer shop. Filters call MedicineService.SearchForCustomers; effective promotional prices appear in the grid. Add to Cart validates through ValidateForCustomerPurchase then CartService.Add and updates the shell cart badge.



```
1  using System;
2  using System.Drawing;
3  using System.Globalization;
4  using System.Linq;
5  using System.Windows.Forms;
6  using System.Windows.Forms.VisualStyles;
7  using SmartMed.Models;
8  using SmartMed.Services;
9
10 namespace SmartMed.UI
11 {
12     public sealed partial class SearchMedicinesForm : EmbeddedPageForm
13     {
14         private readonly MedicineService _medicines;
15         private readonly bool _servicesReady;
16         private bool _runtimeWired;
17         private bool _chromeApplied;
18
19         public SearchMedicinesForm()
20         {
21             InitializeComponent();
22             if (!IsDesignHost())
23             {
24                 _medicines = new MedicineService();
25                 _servicesReady = true;
26             }
27         }
28
29         protected override bool PreferDesignTimePreview() => !_servicesReady || IsDesignHost();
30
31         protected override void OnLoad(EventArgs e)
32         {
33             base.OnLoad(e);
34             ApplyViewChrome();
35             if (_servicesReady)
36                 WireRuntimeBehavior();
37         }
38     }
39 }
```

SmartMed Health Portal - Browse Medicine

HOME BROWSE MEDICINE MY CART MY ORDERS MY PROFILE LOGOUT CONTACT

Browse Catalog

Browse medications and wellness products – add items to your cart.

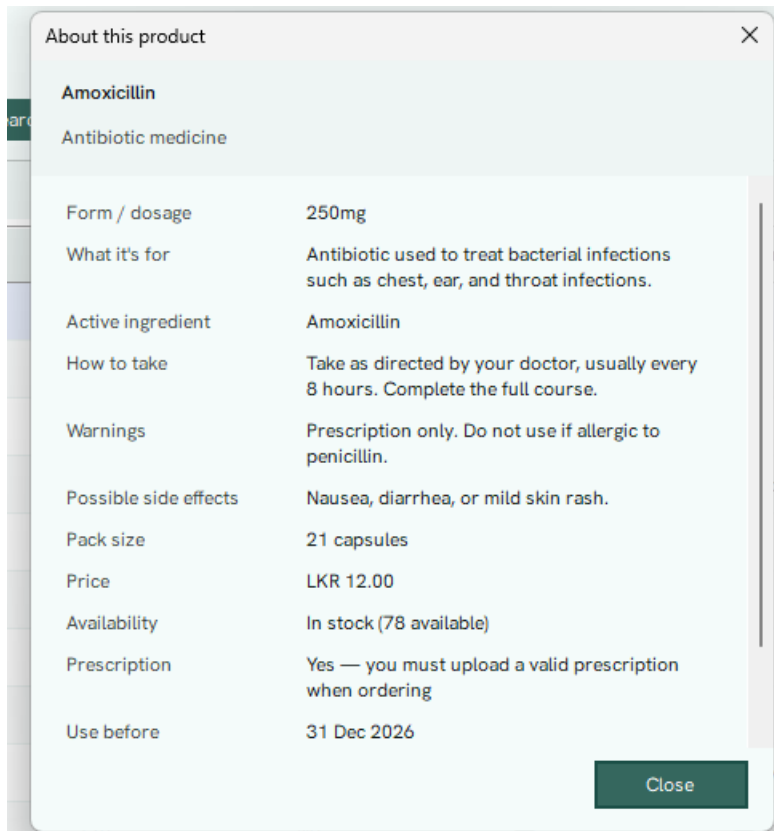
Category: All categories Search Price: Min Max Clear

Medicine Name	Category	Price	Stock Quantity	Rx	Discount	Promo	Qty	Details Btn	Add Btn
Amoxicillin	Antibiotic	LKR 12.00	78	Yes	—	—	1	Details	Add to Cart
Antacid Gel	Gastrointestinal	LKR 8.60	74	No	—	—	1	Details	Add to Cart
Aspirin	Pain Relief	LKR 6.20	180	No	—	—	1	Details	Add to Cart
Atorvastatin	Cardiovascular	LKR 17.96	70	Yes	5%	Discount	1	Details	Add to Cart
Azithromycin	Antibiotic	LKR 22.00	10	Yes	—	—	1	Details	Add to Cart
Betamethasone Cre...	Corticosteroid	LKR 11.80	42	Yes	—	—	1	Details	Add to Cart
Bisoprolol	Cardiovascular	LKR 13.40	78	Yes	—	—	1	Details	Add to Cart
Budesonide Inhaler	Respiratory	LKR 29.50	35	Yes	—	—	1	Details	Add to Cart
Calcium + D3	Supplements	LKR 10.82	130	No	12%	Active	1	Details	Add to Cart
Cetirizine	Antihistamine	LKR 6.50	120	No	—	—	1	Details	Add to Cart
Chlorpheniramine	Allergy	LKR 5.90	140	No	—	—	1	Details	Add to Cart
Ciprofloxacin	Antibiotic	LKR 19.50	45	Yes	—	—	1	Details	Add to Cart

Welcome to SmartMed Secure Session Local Time: 23/2

Medicine details

MedicineDetailsDialog shows full product information before purchase. From SearchMedicinesForm the user clicks Details or double-clicks a row; MedicineService.GetById supplies description, usage, warnings, and related fields.



[Place order \(checkout\)](#)

PlaceOrderForm completes checkout. The customer selects lines, uploads Rx files per item, confirms payment in PaymentCheckoutDialog, and calls OrderService.PlaceOrder, which refreshes prices, checks stock, saves order/items/prescriptions, and reduces stock in one transaction. Success shows order reference #SM-XXXX.

```
1  using System;
2  using System.Drawing;
3  using System.Linq;
4  using System.Windows.Forms;
5  using SmartMed.Services;
6
7  namespace SmartMed.UI
8  {
9      public sealed partial class PlaceOrderForm : EmbeddedPageForm
10     {
11         private readonly OrderService _orders;
12         private readonly MedicineService _medicines;
13         private readonly bool _servicesReady;
14         private bool _runtimeWired;
15         private bool _chromeApplied;
16
17         public PlaceOrderForm()
18         {
19             InitializeComponent();
20             if (!IsDesignHost())
21             {
22                 _orders = new OrderService();
23                 _medicines = new MedicineService();
24                 _servicesReady = true;
25             }
26         }
27
28         protected override bool PreferDesignTimePreview() => !_servicesReady || IsDesignHost();
29
30         protected override void OnLoad(EventArgs e)
31         {
32             base.OnLoad(e);
33             ApplyViewChrome();
34             if (_servicesReady)
35                 WireRuntimeBehavior();
36         }
37
38         protected override void DeRefreshPage() => RefreshCont();
```

SmartMed Health Portal - My Cart

— □ ×

HOMEBROWSE MEDICINEMY CARTMY ORDERSMY PROFILELOGOUTCONTACT

My Cart Checkout

Review items, upload prescriptions, and complete payment.

Shopping Cart

Checko	Medicine Name	Quantity	List Price	Unit Price	Discount Display	Promo Display	Applied	Subtotal	Rx	Prescription	Upload Btn	Preview Btn
--------	---------------	----------	------------	------------	------------------	---------------	---------	----------	----	--------------	------------	-------------

Remove Selected

Clear Cart

Proceed to Payment

Checkout total: LKR 0.00 (0 items)

Track orders

TrackOrdersForm lists the customer's orders and line items. Pending orders can be cancelled with OrderService.CancelOrder. Prescription files open from the grid. Export CSV/PDF saves personal history.

```
TrackOrdersForm.cs X
SmartMed > Forms > Customer > TrackOrdersForm.cs

1  using System;
2  using System.Drawing;
3  using System.Linq;
4  using System.Windows.Forms;
5  using SmartMed.Models;
6  using SmartMed.Services;
7
8  namespace SmartMed.UI
9  {
10     public sealed partial class TrackOrdersForm : EmbeddedPageForm
11     {
12         private readonly OrderService _orders;
13         private readonly MedicineService _medicines;
14         private readonly bool _servicesReady;
15         private bool _runtimeWired;
16         private bool _chromeApplied;
17         private int? _selectedOrderId;
18
19         public TrackOrdersForm()
20         {
21             InitializeComponent();
22             if (!IsDesignHost())
23             {
24                 _orders = new OrderService();
25                 _medicines = new MedicineService();
26                 _servicesReady = true;
27             }
28         }
29
30         protected override bool PreferDesignTimePreview() => !_servicesReady || IsDesignHost();
31
32         protected override void OnLoad(EventArgs e)
33         {
34             base.OnLoad(e);
35             ApplyViewChrome();
36             if (_servicesReady)
37                 WireRuntimeBehavior();
38         }
39     }
40 }
```

SmartMed Health Portal - My Orders — □ ×

HOME BROWSE MEDICINE MY CART **MY ORDERS** MY PROFILE LOGOUT CONTACT

My Orders

View order history, cancel pending orders, and export receipts.

Cancel Pending Order Export CSV Export PDF

Your Orders						
Order Ref	Order Date	Status	Total	Payment	Prescription	Cancel Reason
#SM-0016	Jul 14, 2026 10:15 PM	Ready for Pickup	LKR 12.00	Cash on Pickup (Pay on Pic...	3_2_20260714221507850...	—
#SM-0015	Jul 14, 2026 10:13 PM	Cancelled	LKR 8.60	Cash on Pickup (Pay on Pic...	—	No Need
#SM-0013	Jul 14, 2026 08:59 PM	Cancelled	LKR 22.00	Cash on Pickup (Pay on Pic...	3_18_2026071420594405...	No stock
#SM-0012	Jul 14, 2026 08:59 PM	Pending	LKR 9.00	Cash on Pickup (Pay on Pic...	—	—
#SM-0011	Jul 14, 2026 08:48 PM	Pending	LKR 10.00	Cash on Pickup (Pay on Pic...	—	—
#SM-0008	Jul 12, 2026 09:16 PM	Pending	LKR 8.60	Cash on Pickup (Pay on Pic...	—	—

Order Items				
Medicine Name	Quantity	Unit Price	Discount	Subtotal
Amoxicillin	1	LKR 12.00	—	LKR 12.00

Welcome to SmartMed Secure Session Local Time: 23:27

Profile management

ProfileManagementForm updates customer contact details. LoadProfile reads Session.CurrentCustomer; BtnSaveProfile_Click calls CustomerService.UpdateProfile and refreshes Session. BtnChangePassword opens ChangePasswordForm. Email remains the login identity.

```
ProfileManagementForm.cs X
SmartMed > Forms > Customer > ProfileManagementForm.cs

1  using System;
2  using System.Drawing;
3  using System.Windows.Forms;
4  using SmartMed.Models;
5  using SmartMed.Services;
6
7  namespace SmartMed.UI
8  {
9      public sealed partial class ProfileManagementForm : EmbeddedPageForm
10     {
11         private readonly CustomerService _customers;
12         private readonly bool _servicesReady;
13         private bool _runtimeWired;
14         private bool _chromeApplied;
15
16         public ProfileManagementForm()
17         {
18             InitializeComponent();
19             if (!IsDesignHost())
20             {
21                 _customers = new CustomerService();
22                 _servicesReady = true;
23             }
24         }
25
26         protected override bool PreferDesignTimePreview() => !_servicesReady || IsDesignHost();
27
28         protected override void OnLoad(EventArgs e)
29         {
30             base.OnLoad(e);
31             ApplyViewChrome();
32             if (_servicesReady)
33                 WireRuntimeBehavior();
34         }
35
36         protected override void DoRefreshPage() => LoadProfile();
37     }
```

SmartMed Health Portal - My Profile

HOMEBROWSE MEDICINEMY CARTMY ORDERSMY PROFILELOGOUTCONTACT

My Profile

Update your contact details and password.

PERSONAL DETAILS

Full Name *

Nisha

Email *

customer@gmail.com

Phone *

0771112233

Address *

10 Demo Lane, Colombo 05

Save Profile

Change Password

Welcome to SmartMedSecure SessionLocal Time: 23:27

Search Algorithms

SmartMed uses linear (sequential) search for all in-memory catalogue and grid lookups. The algorithm is implemented in `SearchService.cs`. It walks a list from the first element to the last and adds matching items to a result list. In the worst case every item is examined, so the time complexity is $O(n)$ for a list of n records.

Linear search was chosen because pharmacy catalogues are typically modest in size, and users need partial text matching (for example finding "Para" inside "Paracetamol"). Binary search would require a sorted collection and exact key comparison; it is unsuitable for case-insensitive substring matching on medicine names, categories, or customer contact fields without rebuilding sorted indexes for every filter.

`SearchService.SearchByName` compares each `MedicineName` with `String.Contains` after normalising to lower case. `FilterByCategory` and `FilterByPriceRange` apply the same sequential style for category text and price bounds. Search combines these passes in order: name, then category, then price. `SearchCustomers` applies the same linear pattern across customer identifier,

name, email, and phone. SearchOrders provides an equivalent linear scan over order references, customer name, and status.

MedicineService.Search and SearchForCustomers call SearchService.Search so both admin Manage Medicines and customer Search Medicines screens share one algorithm.

CustomerService uses SearchCustomers for admin customer lookup. This single service keeps the coursework search requirement clear, testable, and documented in one place.

```

public static List<Medicine> Search(List<Medicine> medicines, string name, string category)
{
    // Narrow the medicine list by name, category, and price in that order.
    var results = medicines;
    if (!string.IsNullOrEmpty(name))
        results = SearchByName(results, name);
    if (!string.IsNullOrEmpty(category))
        results = FilterByCategory(results, category);
    if (minPrice.HasValue || maxPrice.HasValue)
    {
        decimal min = minPrice ?? 0;
        decimal max = maxPrice ?? decimal.MaxValue;
        results = FilterByPriceRange(results, min, max);
    }
    return results;
}

```

```

public static List<Customer> SearchCustomers(IList<Customer> customers, string keyword)
{
    if (string.IsNullOrEmpty(keyword))
        return new List<Customer>(customers);

    // Find customers by ID, name, email, or phone number.
    var key = keyword.Trim();
    var keyDigits = ValidationService.ToSriLankaLocalDigits(key);
    if (keyDigits.Length == 0)
        keyDigits = ValidationService.NormalizePhoneDigits(key);
    var results = new List<Customer>();

    foreach (var customer in customers)
    {
        if (MatchesCustomerId(customer, key))
        {
            results.Add(customer);
            continue;
        }
        if (customer.Name != null
            && customer.Name.IndexOf(key, StringComparison.OrdinalIgnoreCase) >= 0)
        {
            results.Add(customer);
            continue;
        }
        if (customer.Email != null
            && customer.Email.IndexOf(key, StringComparison.OrdinalIgnoreCase) >= 0)
        {
            results.Add(customer);
            continue;
        }
    }
}

```

User Manual

Installation Guide

What you need before you start

- A Windows 10 or later computer
- .NET Framework 4.8
- Microsoft SQL Server (Express, LocalDB, or full version)
- Visual Studio 2019 or later -only if building from source code

Step 1 -Create the database

1. Open SQL Server Management Studio.
2. Connect to your SQL Server instance.
3. Open Database/SmartMedDB.sql from the project folder.
4. Click Execute and wait until finished.
5. Check that database SmartMedDB exists with tables and sample data.

Step 2 -Set the connection string

1. Open SmartMed/App.config.
2. Find the SmartMedDB connection string.
3. Set Data Source to your SQL Server name.
4. Save the file.

Step 3 -Build and run

1. Open SmartMed.sln in Visual Studio.
2. Set SmartMed as startup project.
3. Build (Ctrl+Shift+B), then run (F5).
4. The login screen should open.

Step 4 -Test logins

- Admin: admin / admin123
- Customer: customer@gmail.com / Customer123 (sample data)

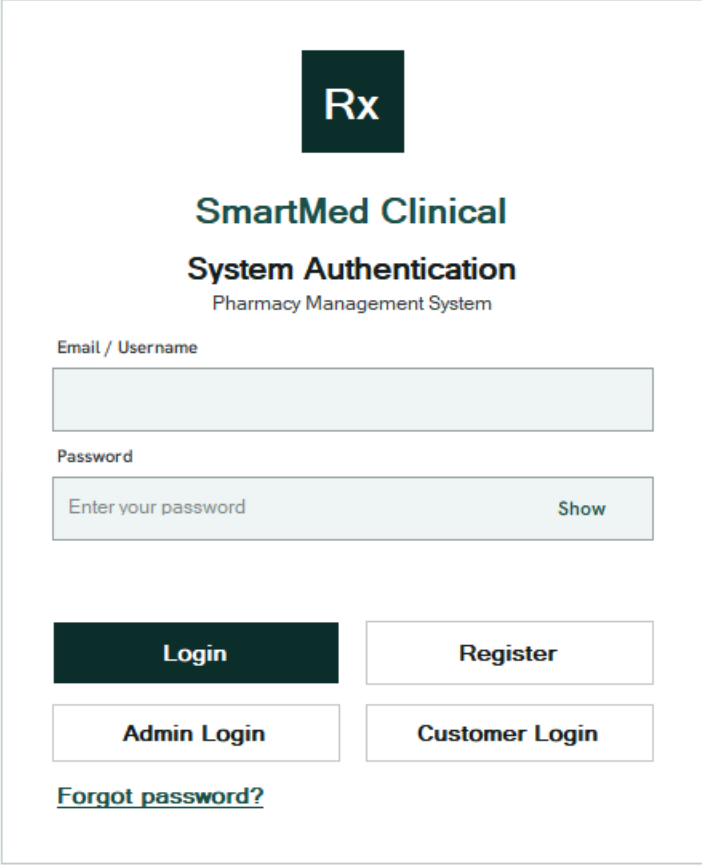
If you see a database warning, check Step 2 and confirm SQL Server is running.

Using the Application

This guide walks a new user through SmartMed step by step. Follow each section in order the first time you use the system. Screenshot numbers (U1, A1, C1, etc.) match the Interface screenshot labels under each function in the Implementation section.

PART 1 -Starting the application

1. Double-click SmartMed.exe (or press F5 in Visual Studio after building).
2. Wait for the login window to appear. If a database warning shows, contact your administrator -the app cannot save data until the database is connected.



The image shows a login and registration form for the SmartMed Clinical System. At the top is a dark green square logo with the white text 'Rx'. Below the logo, the text 'SmartMed Clinical' is in a large, bold, dark green font, followed by 'System Authentication' in a bold black font, and 'Pharmacy Management System' in a smaller black font. The form contains two input fields: 'Email / Username' and 'Password'. The 'Password' field has a 'Show' button next to it. Below the input fields are four buttons: 'Login' (dark green with white text), 'Register' (white with dark green text), 'Admin Login' (white with dark green text), and 'Customer Login' (white with dark green text). At the bottom, there is a link that says 'Forgot password?' in a dark green font.

Rx

SmartMed Clinical
System Authentication
Pharmacy Management System

Email / Username

Password

Enter your password [Show](#)

Login **Register**

Admin Login **Customer Login**

[Forgot password?](#)

PART 2 -Common tasks (both roles)

2A. Logging in as Admin

1. On the login screen, select the Admin option.
2. Type username: admin (or your admin username).
3. Type password.
4. Click Login.
5. The admin home screen opens with the sidebar on the left.

2B. Logging in as Customer

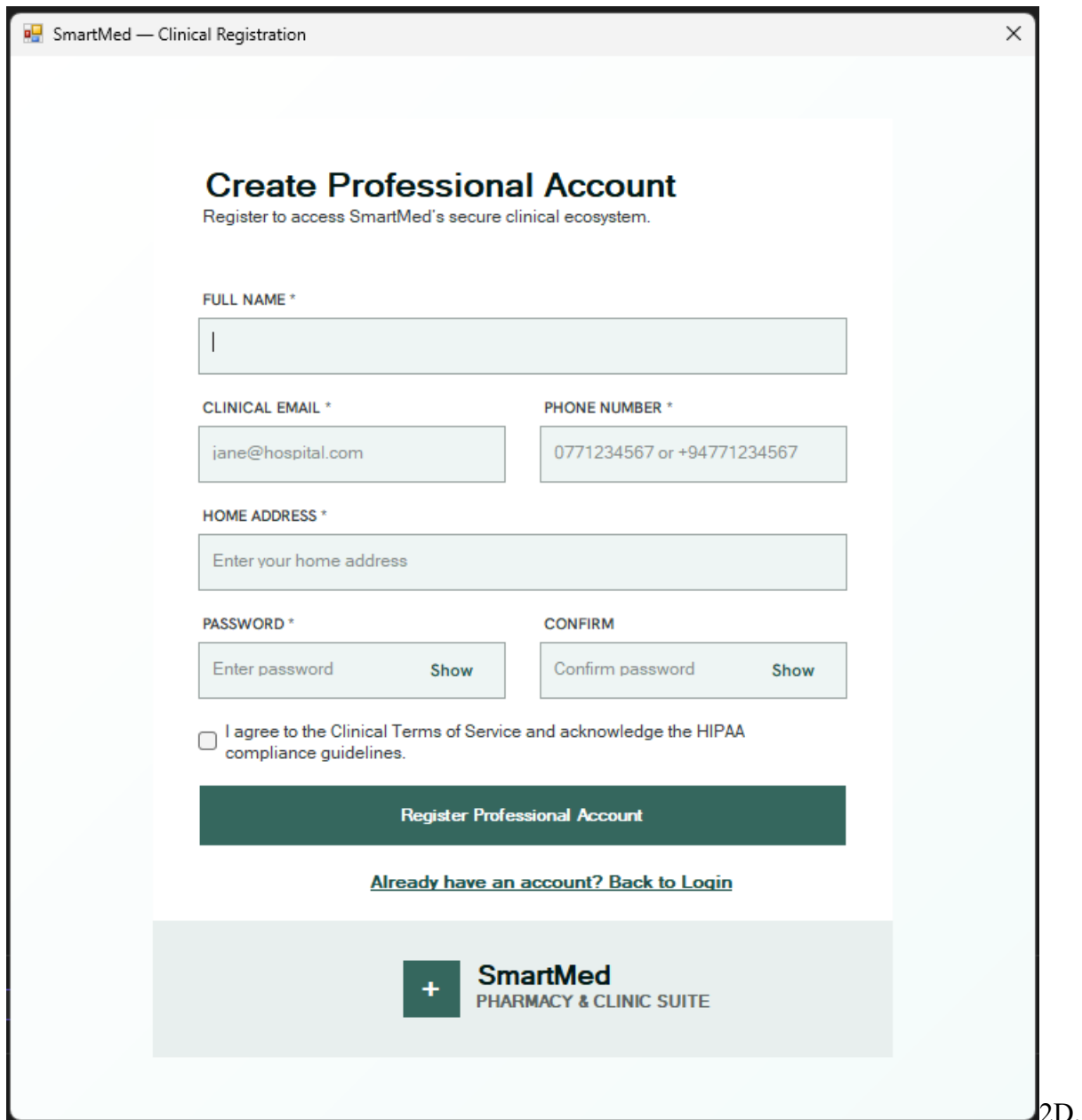
1. Select Customer on the login screen.

2. Enter your registered email and password.
3. Click Login.
4. The customer home screen opens.

If you do not have an account, use Register first (section 2C).

2C. Registering a new customer account

1. From the login screen, click Register.
2. Fill in full name, email, phone, address, password, and confirm password.
3. Click Register / Save.
4. When a success message appears, return to login and sign in with your new email and password.



The image shows a web browser window titled "SmartMed — Clinical Registration". The main heading is "Create Professional Account" with a subtext "Register to access SmartMed's secure clinical ecosystem." The form contains several input fields: "FULL NAME *" (empty), "CLINICAL EMAIL *" (containing "jane@hospital.com"), "PHONE NUMBER *" (containing "0771234567 or +94771234567"), "HOME ADDRESS *" (containing "Enter your home address"), "PASSWORD *" (containing "Enter password" and a "Show" button), and "CONFIRM" (containing "Confirm password" and a "Show" button"). Below these fields is a checkbox labeled "I agree to the Clinical Terms of Service and acknowledge the HIPAA compliance guidelines." followed by a dark green button labeled "Register Professional Account". At the bottom, there is a link "Already have an account? Back to Login" and a footer with a green square containing a white plus sign, followed by the text "SmartMed PHARMACY & CLINIC SUITE".

SmartMed — Clinical Registration

Create Professional Account

Register to access SmartMed's secure clinical ecosystem.

FULL NAME *

CLINICAL EMAIL *

PHONE NUMBER *

HOME ADDRESS *

PASSWORD *

CONFIRM

☐ I agree to the Clinical Terms of Service and acknowledge the HIPAA compliance guidelines.

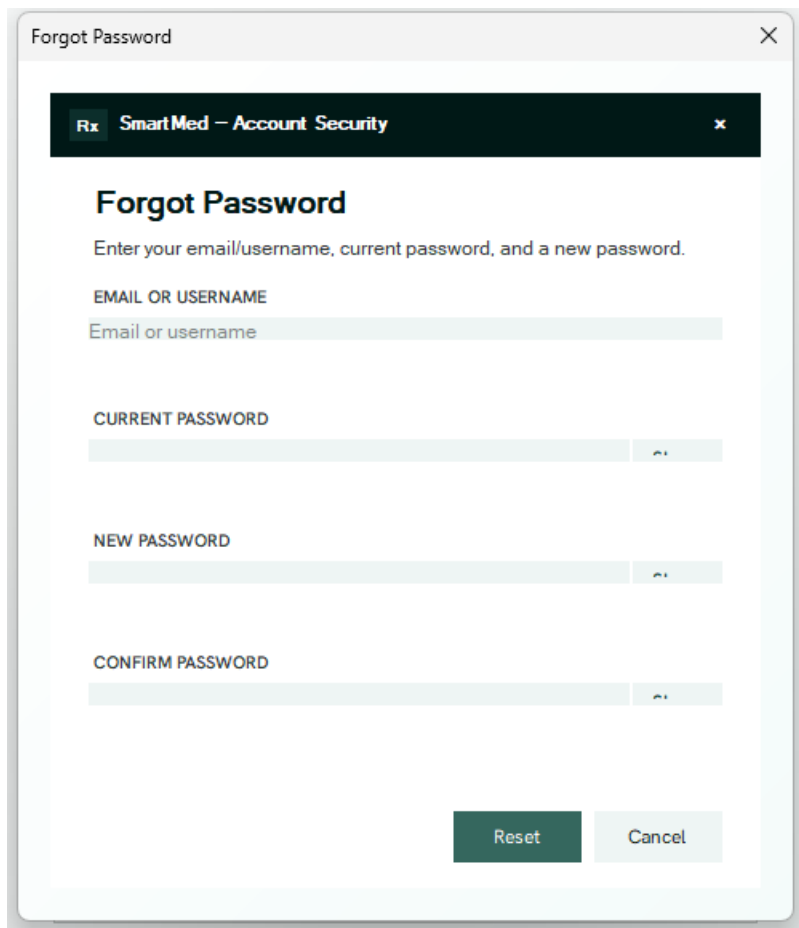
Register Professional Account

[Already have an account? Back to Login](#)

 **SmartMed**
PHARMACY & CLINIC SUITE

Forgot password

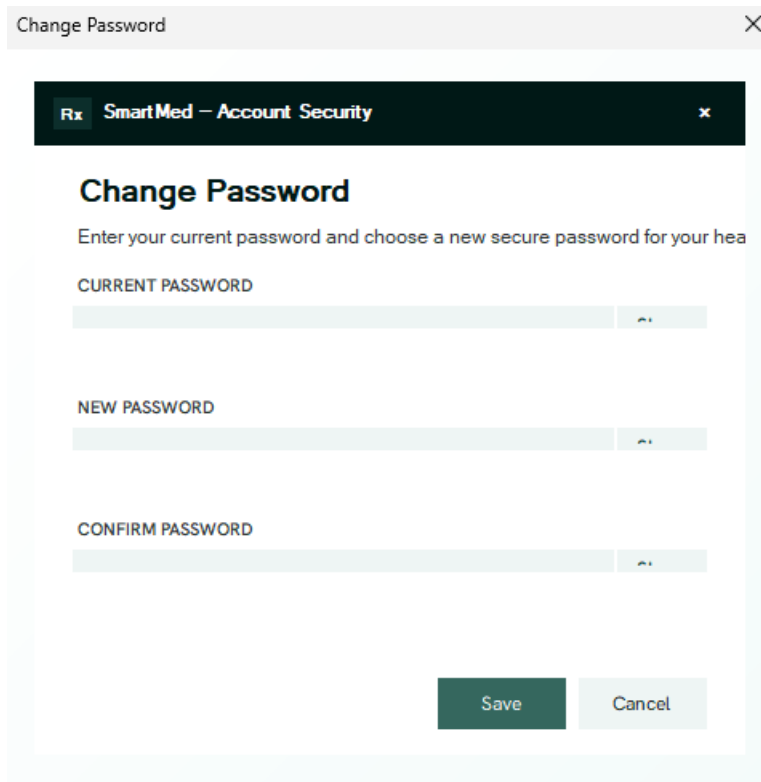
1. From login, click Forgot password.
2. Enter the email address for your account.
3. Follow the on-screen steps to set a new password.
4. Return to login and sign in again.



The image shows a 'Forgot Password' dialog box for the SmartMed application. The dialog has a title bar with 'Forgot Password' and a close button. Inside, there is a dark header bar with 'Rx SmartMed – Account Security' and a close button. The main content area is white and contains the title 'Forgot Password' followed by the instruction 'Enter your email/username, current password, and a new password.' Below this are three input fields: 'EMAIL OR USERNAME' with a placeholder 'Email or username', 'CURRENT PASSWORD', and 'NEW PASSWORD'. Each password field has a toggle icon to its right. At the bottom, there are two buttons: 'Reset' (dark green) and 'Cancel' (light green).

2E. Change password (after login)

1. Open Change Password from the admin or customer menu.
2. Enter current password, new password, and confirm new password.
3. Click Save. Use the new password next time you log in.



The image shows a 'Change Password' dialog box. At the top, there is a title bar with the text 'Change Password' and a close button (X). Below the title bar, there is a dark header bar with the text 'Rx SmartMed – Account Security' and a close button (X). The main content area has the title 'Change Password' and a subtitle 'Enter your current password and choose a new secure password for your head'. There are three input fields: 'CURRENT PASSWORD', 'NEW PASSWORD', and 'CONFIRM PASSWORD'. Each field has a password strength indicator on the right. At the bottom, there are two buttons: 'Save' and 'Cancel'.

2F. Logging out

1. Click Logout in the sidebar or close button on the top bar.
2. You return to the login screen. Your session is cleared.

PART 3 -Admin user guide

3A. Admin dashboard (Overview)

1. After admin login, Overview opens automatically.
2. Read the summary cards: total sales, medicines in stock, active orders.
3. Check expiry and low-stock alerts if shown -these warn you before products run out or expire.
4. Use the sidebar to go to other admin pages.

SmartMed Clinical Management - Operational Dashboard

— □ ×

DASHBOARD MEDICINE CUSTOMERS ORDERS (7) REPORTS LOGOUT **A ADMIN**
SYSTEM ADMIN

Operational Dashboard

Overview of pharmaceutical stock and fulfillment health. [Refresh](#)

STOCK ITEMS
56

PENDING ORDERS
7

REVENUE (LKR)
33

ACTIVE USERS
8

CRITICAL ALERTS

Medicine Name	Level	Status
Azithromycin	10 units	LOW

EXPIRY WARNINGS

Batch Id	Medicine	Due Date
#M-005	Cough Syrup	27 Days

RECENT FULFILLMENT ACTIVITY [Print](#)

Order Ref	Customer Name	Fulfillment Status	Timestamp	Actions
#SM-0016	Nisha	Ready for Pickup	1 hour ago	Open
#SM-0015	Nisha	Cancelled	1 hour ago	Open
#SM-0014	Test Customer	Cancelled	1 hour ago	Open
#SM-0013	Nisha	Cancelled	2 hours ago	Open

3B. Manage medicines -add a new product

1. Click Medicines in the sidebar.
2. Click Add (or clear the form for a new entry).
3. Enter medicine name, category, dosage, price, stock quantity, supplier, expiry date.
4. Tick Requires Prescription if customers must upload a script.
5. Set promotion dates and discount if needed.
6. Click Save. The new row appears in the grid.

Add Medicine

×

Medicine Name *

Category *

Antibiotic

▼

Dosage / Form *

Unit Price (LKR) *

Stock *

Expiry Date *

14-Jan-27

▼

Supplier *

Discount % (optional)

☐ Requires Prescription (Rx)

☐ On Promotion

Promotion Start

14-Jul -26

▼

Promotion End

14-Jul -26

▼

Description (What it's for)

▲

▼

Active Ingredient

Usage Instructions (How to take)

▲

▼

Add

Cancel

3C. Manage medicines -edit or deactivate

1. Click a row in the medicine grid.
2. Change fields in the form on the right (or below).
3. Click Update to save changes.
4. To remove from the shop (without deleting history), click Deactivate. Confirm when asked.
5. If the system blocks deactivation, an open order still references that medicine -finish or cancel that order first.

Edit Medicine

×

Medicine Name *

Amoxicillin

Category *

Antibiotic

Dosage / Form *

250mg

Unit Price (LKR) *

12.00

Stock *

78

Expiry Date *

31-Dec-26

Supplier *

MediSupply

Discount % (optional)

☒ Requires Prescription (Rx)

☐ On Promotion

Promotion Start

14-Jul-26

Promotion End

14-Jul-26

Description (What it's for)

Antibiotic used to treat bacterial infections such as chest, ear, and throat infections.

Active Ingredient

Amoxicillin

Usage Instructions (How to take)

Take as directed by your doctor, usually every 8 hours. Complete the full course.

Update

Cancel

Confirm Deactivate

×



Deactivate this medicine? It will be hidden from customers. Order history is kept.

Yes

No

3D. Manage customers

1. Click Customers in the sidebar.
2. Use search to find a customer.
3. Click a row to load details.
4. Edit name, phone, or address and click Save.
5. Use Activate / Deactivate to block a customer from logging in without deleting their order history.

SmartMed Clinical Management - Manage Customers

— □ ×

DASHBOARD MEDICINE CUSTOMERS ORDERS (7) REPORTS LOGOUT ADMIN SYSTEM ADMIN

Manage Customers

View customer records. Order Activity shows recent ordering; use Account to enable or disable login.

Export Print

TOTAL CUSTOMERS
8

ENABLED
8

DISABLED
0

+ Add Customer Reload Search:

Customer ID	Name	Contact	Last Order	Orders	Order Activity	Edit	Account	Remove
#SM-00009	Test Customer	test@gmail.com / 07564...	2026-07-14	1	ACTIVE	Edit	Deactivate	Remove
#SM-00008	Jane	john2@email.com / 072...	—	0	INACTIVE	Edit	Deactivate	Remove
#SM-00006	Ruwani Fernando	ruwan.fernando@gmail...	—	0	INACTIVE	Edit	Deactivate	Remove
#SM-00005	Kamala Silva	kamala.silva@gmail.com...	2026-07-08	1	ACTIVE	Edit	Deactivate	Remove
#SM-00004	Nimal Perera	nimal.perera@gmail.co...	2026-07-06	1	ACTIVE	Edit	Deactivate	Remove
#SM-00003	Nisha	customer@gmail.com / ...	2026-07-14	8	ACTIVE	Edit	Deactivate	Remove

8 customers

3E. Manage orders -update status

1. Click Orders in the sidebar.
2. Select an order from the list.
3. View line items and total in the detail area.
4. Choose a new status: Pending, Ready for Pickup, or Delivered.
5. Click Update Status.

6. For orders with prescriptions, open the prescription file and click Verify or Reject before releasing medicine.

SmartMed Clinical Management - Manage Orders

DASHBOARD MEDICINE CUSTOMERS **ORDERS (7)** REPORTS LOGOUT ADMIN SYSTEM ADMIN

Manage Orders

Monitor and process pharmaceutical orders across all clinical departments.

VOLUME
14

AVG TIME
239.6h

FLAGS
00

Filter Options

Search Orders: Status Category: Service Date Range: ☐ Use date range 14-Jul-26 to 14-Jul-26

Order Ref	Customer	Order Date	Total Amount	Prescription	Rx Status	Status	Cancel Reason	Verify	Reject	Cancel	View
#ORD-0016	Nisha (PAT-003-...	Jul 14, 2026 22...	LKR 12.00	3_2_2026071422...	Verified	Ready for Pl...	—	Verify	Reject	Cancel	View
#ORD-0015	Nisha (PAT-003-...	Jul 14, 2026 22...	LKR 8.60	—	—	Pending	No Need	Verify	Reject	Cancel	View
#ORD-0014	Test Customer (...)	Jul 14, 2026 21...	LKR 12.00	9_2_2026071421...	Pending	Pending	Test	Verify	Reject	Cancel	View
#ORD-0013	Nisha (PAT-003-...	Jul 14, 2026 20...	LKR 22.00	3_18_202607142...	Pending	Pending	No stock	Verify	Reject	Cancel	View
#ORD-0012	Nisha (PAT-003-...	Jul 14, 2026 20...	LKR 9.00	—	—	Pending	—	Verify	Reject	Cancel	View

<

3F. Reports

1. Click Reports in the sidebar.
2. Switch between Sales, Stock, and Customer history tabs (if available).
3. Read summary figures for management decisions.
4. Export or print if your build supports it.

SmartMed Clinical Management - Generate Reports

DASHBOARD MEDICINE CUSTOMERS ORDERS (7) **REPORTS** LOGOUT

ADMIN
SYSTEM ADMIN

Reports

Completed sales, stock alerts, and customer order history for quick decisions.

[View Report](#) [Export CSV](#) [Export PDF](#)

COMPLETED REVENUE
LKR 0.00

COMPLETED ORDERS
0

LOW STOCK ITEMS
1

OUTSTANDING
LKR 138.03

Sales Performance Medicine Inventory Customer Order History

Period: **Week** Month Year ☐ Use date range 08-Jul-26 to 14-Jul-26

Sales Performance Report

Period: Week | Completed orders only | Generated Jul 14, 2026 11:36 PM

Order Ref	Customer	Order Date	Total	Status
-----------	----------	------------	-------	--------

Items: 0 | Completed sales only | Week | 11:36 PM | Jul 14, 2026

PART 4 -Customer user guide

4A. Customer home

1. After customer login, Home shows a welcome summary.
2. Note promotions or cart reminders.
3. Use Browse to shop or Orders to track purchases.

SmartMed Health Portal - Home

— □ ×

HOMEBROWSE MEDICINEMY CARTMY ORDERSMY PROFILELOGOUTCONTACT

Customer Home

Welcome, Nisha – browse medicines, manage your cart, and track orders.

ITEMS IN CART
0

ACTIVE ORDERS
8

PROMOTIONS
4

Recent Orders

Order Ref	Order Date	Status	Total
#SM-0016	14-Jul-26 10:15 PM	Ready for Pickup	LKR 12.00
#SM-0015	14-Jul-26 10:13 PM	Cancelled	LKR 8.60
#SM-0013	14-Jul-26 8:59 PM	Cancelled	LKR 22.00
#SM-0012	14-Jul-26 8:59 PM	Pending	LKR 9.00
#SM-0011	14-Jul-26 8:48 PM	Pending	LKR 10.00

4B. Search and browse medicines

1. Click Browse in the sidebar.
2. Type part of a medicine name in the search box, or pick a category and price range.
3. Click Search or press Enter.
4. Click a row or Details to read full product information.
5. Enter quantity and click Add to Cart.

SmartMed Health Portal - Browse Medicine

HOMEBROWSE MEDICINEMY CARTMY ORDERSMY PROFILELOGOUTCONTACT

Browse Catalog

Browse medications and wellness products – add items to your cart.

Category: All categories Search Price: Min Max Clear

Medicine Name	Category	Price	Stock Quantity	Rx	Discount	Promo	Qty	Details Btn	Add Btn
Amoxicillin	Antibiotic	LKR 12.00	78	Yes	—	—	1	Details	Add to Cart
Antacid Gel	Gastrointestinal	LKR 8.60	74	No	—	—	1	Details	Add to Cart
Aspirin	Pain Relief	LKR 6.20	180	No	—	—	1	Details	Add to Cart
Atorvastatin	Cardiovascular	LKR 17.96	70	Yes	5%	Discount	1	Details	Add to Cart
Azithromycin	Antibiotic	LKR 22.00	10	Yes	—	—	1	Details	Add to Cart
Betamethasone Cre...	Corticosteroid	LKR 11.80	42	Yes	—	—	1	Details	Add to Cart
Bisoprolol	Cardiovascular	LKR 13.40	78	Yes	—	—	1	Details	Add to Cart
Budesonide Inhaler	Respiratory	LKR 29.50	35	Yes	—	—	1	Details	Add to Cart
Calcium + D3	Supplements	LKR 10.82	130	No	12%	Active	1	Details	Add to Cart
Cetirizine	Antihistamine	LKR 6.50	120	No	—	—	1	Details	Add to Cart
Chlorpheniramine	Allergy	LKR 5.90	140	No	—	—	1	Details	Add to Cart
Ciprofloxacin	Antibiotic	LKR 19.50	45	Yes	—	—	1	Details	Add to Cart

About this product

Amoxicillin

Antibiotic medicine

Form / dosage	250mg
What it's for	Antibiotic used to treat bacterial infections such as chest, ear, and throat infections.
Active ingredient	Amoxicillin
How to take	Take as directed by your doctor, usually every 8 hours. Complete the full course.
Warnings	Prescription only. Do not use if allergic to penicillin.
Possible side effects	Nausea, diarrhea, or mild skin rash.
Pack size	21 capsules
Price	LKR 12.00
Availability	In stock (78 available)
Prescription	Yes — you must upload a valid prescription when ordering
Use before	31 Dec 2026

Close

4C. Cart and checkout

1. Click Cart in the sidebar.
2. Review each line: name, quantity, price, and total.
3. For prescription (Rx) items, click the prescription column and upload a PDF or image file for each Rx line.
4. Tick which items you want to buy if some lines are unchecked.
5. Click Place Order.
6. In the payment dialog, choose payment method and confirm the total.
7. Click Confirm. A success message shows your order number.

SmartMed Health Portal - My Cart

HOMEBROWSE MEDICINEMY CARTMY ORDERSMY PROFILELOGOUTCONTACT

My Cart Checkout

Review items, upload prescriptions, and complete payment.

Shopping Cart												
Checko	Medicine Name	Quantity	List Price	Unit Price	Discount Display	Promo Display	Applied	Subtotal	Rx	Prescription	Upload Btn	Preview Btn
☒	Amoxicillin	1	LKR 12.00	LKR 12.00	—	—	—	LKR 12.00	Yes	Required	Upload Rx	Preview

Remove SelectedClear CartProceed to Payment

Checkout total: LKR 12.00 (1 items)

Rx medicines require a prescription upload.

Welcome to SmartMedSecure SessionLocal Time: 23:36

Payment

×

Complete Payment

Amount payable: LKR 12.00

Payment method

☒ Card

☐ Cash on Pickup

☐ Bank Transfer

Card Number *

Enter 16-digit card number

Cardholder Name *

Name as printed on card

Expiry Date (MM/YY) *

MM/YY

CVV / Security Code *

3 or 4 digits

Your card details are validated locally for this academic demo. No real charge is made.

Cancel

Pay Place Order

4D. Track orders

1. Click Orders in the sidebar.
2. Select your order from the list.
3. Read status: Pending, Ready for Pickup, Delivered, or Cancelled.
4. View line items and amounts.
5. To cancel, select a Pending order, click Cancel, and enter a reason.
6. To export history, use Export PDF or Export CSV if shown.

SmartMed Health Portal - My Orders

HOMEBROWSE MEDICINEMY CARTMY ORDERSMY PROFILELOGOUTCONTACT

My Orders

View order history, cancel pending orders, and export receipts.

Cancel Pending OrderExport CSVExport PDF

Order Ref	Order Date	Status	Total	Payment	Prescription	Cancel Reason
#SM-0016	Jul 14, 2026 10:15 PM	Ready for Pickup	LKR 12.00	Cash on Pickup (Pay on Pic...	3_2_20260714221507850...	—
#SM-0015	Jul 14, 2026 10:13 PM	Cancelled	LKR 8.60	Cash on Pickup (Pay on Pic...	—	No Need
#SM-0013	Jul 14, 2026 08:59 PM	Cancelled	LKR 22.00	Cash on Pickup (Pay on Pic...	3_18_2026071420594405...	No stock
#SM-0012	Jul 14, 2026 08:59 PM	Pending	LKR 9.00	Cash on Pickup (Pay on Pic...	—	—
#SM-0011	Jul 14, 2026 08:48 PM	Pending	LKR 10.00	Cash on Pickup (Pay on Pic...	—	—
#SM-0008	Jul 12, 2026 09:16 PM	Pending	LKR 8.60	Cash on Pickup (Pay on Pic...	—	—

Medicine Name	Quantity	Unit Price	Discount	Subtotal
Amoxicillin	1	LKR 12.00	—	LKR 12.00

4E. Update profile

1. Click Profile in the sidebar.
2. Edit name, phone, or address.
3. Click Save.
4. Changes apply to future orders and contact details.

SmartMed Health Portal - My Profile

HOME BROWSE MEDICINE MY CART MY ORDERS MY PROFILE LOGOUT CONTACT

My Profile

Update your contact details and password.

PERSONAL DETAILS

Full Name *
Nisha

Email *
customer@gmail.com

Phone *
0771112233

Address *
10 Demo Lane, Colombo 05

Save Profile Change Password

PART 5 -Troubleshooting

- Login fails -check username/email and password; customers must register first.
- Checkout fails -check stock, upload all required prescriptions, and ensure medicines are still available.
- Admin cannot deactivate medicine -order still pending or ready; complete or cancel it first.
- Database error -ask administrator to check SQL Server and App.config connection string.

Reflection

When I started SmartMed, my main goal was to move from small classroom exercises to one complete desktop application that a pharmacy could actually use. I did not want only a login form and one table -I wanted stock, orders, customers, and reports to work together. That meant I had to plan more carefully than in earlier modules.

The part I found most useful was splitting the project into folders for forms, services, data access, and models. At first I was tempted to put SQL inside model classes, like in a basic

student CRUD example, but I realised that would become messy once prescriptions, promotions, and order status were added. Keeping rules in service classes made debugging easier. When checkout pricing was wrong, I could follow the flow from PlaceOrderForm to OrderService and see where the price was calculated.

I also learned that design diagrams are not just for the report -they helped me before coding. The use case diagram reminded me to finish customer features and not only admin screens. The database diagram stopped me from deleting order history when a medicine is removed, which was an important business rule I might have missed.

Not everything was smooth. Connecting to SQL Server on my own laptop took several attempts because the connection string in App.config had to match my instance name. WinForms layout was another challenge: admin pages are built in code for the shell form, so I had to test both the Visual Studio designer and runtime to make sure screens looked right.

If I had more time, I would add automated tests for services and email alerts when order status changes. Even so, I am satisfied with the result. I understand layered design better now, and I can explain my choices clearly in a viva. Overall, SmartMed helped me connect OOP theory, database design, and real user requirements in one project.

Conclusion

SmartMed is a working pharmacy management application for Windows that supports both administrator and customer roles. It covers inventory, orders, prescriptions, dashboards, and reports using C# WinForms, .NET Framework 4.8, and SQL Server.

The project meets the coursework requirements through documented functional and non-functional needs, UML-style diagrams, a clear class and service structure, implementation explanation, and user instructions. Security and data integrity were considered through hashed passwords, validated inputs, and careful order and stock handling.

The solution can be installed from the provided database script, demonstrated with sample logins, and extended in future work without redesigning the whole architecture. It represents a practical application of object-oriented and database concepts learned in Application Development.